



UNIVERSIDAD DE CASTILLA-LA MANCHA

ESCUELA SUPERIOR DE INFORMÁTICA
DEPARTAMENTO DE TECNOLOGÍAS Y SISTEMAS DE LA INFORMACIÓN

ESCUELA SUPERIOR DE INGENIERÍA INFORMÁTICA
DEPARTAMENTO DE SISTEMAS INFORMÁTICOS

Tareas hardware para FreeRTOS y X-Heep

Máster en Sistemas Microelectrónicos
Basados en Arquitecturas Abiertas

Autor:

Antonio Fernando Mateo Francés

Director:

Fernando Rincón Calle

Ciudad Real, Albacete, 2025

Tareas hardware para FreeRTOS y X-Heep

Máster en Sistemas Microelectrónicos Basados en Arquitecturas Abiertas

Financiado por el Ministerio de Transformación Digital y Función Pública de España a través del programa PERTE Chip (Cátedra UCLM, TSI-069100-2023-0014).



ESCUELA SUPERIOR DE INFORMÁTICA
DEPARTAMENTO DE TECNOLOGÍAS Y SISTEMAS DE LA INFORMACIÓN

ESCUELA SUPERIOR DE INGENIERÍA INFORMÁTICA
DEPARTAMENTO DE SISTEMAS INFORMÁTICOS

UNIVERSIDAD DE CASTILLA-LA MANCHA

Tareas hardware para FreeRTOS y X-Heep

Máster en Sistemas Microelectrónicos
Basados en Arquitecturas Abiertas

Realizada por:

ANTONIO FERNANDO MATEO FRANCÉS

Ingeniero Informático
Universidad de Castilla-La Mancha

Dirigida por:

FERNANDO RINCÓN CALLE

Profesor Titular de Universidad
Universidad de Castilla-La Mancha

Ciudad Real, Albacete, 2025

Antonio Fernando Mateo Francés

Escuela Superior de Informática
Edificio Fermín Caballero
Paseo de la Universidad, 4
13071 Ciudad Real, España

Escuela Superior de Ingeniería Informática
Edificio Infante Don Juan Manuel
Avda. de España s/n.
02071 Albacete, España

Teléfono: 648556069

E-mail: `antoniofernando.mateo@alu.uclm.es`

Web site:

© Antonio Fernando Mateo Francés, 2025.

Se permite la copia, distribución y/o modificación de este documento bajo los términos de la Licencia CC BY-NC-SA 4.0. El texto completo de la licencia puede obtenerse en <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

Este documento fue realizado en L^AT_EX.

TRIBUNAL:

Presidente: _____

Vocal: _____

Secretario(a): _____

FECHA DE DEFENSA: _____

CALIFICACIÓN: _____

PRESIDENTE

VOCAL

SECRETARIO(A)

Fdo.:

Fdo.:

Fdo.:

Agradecimientos

Primero, quiero dar las gracias a mi familia, que me ha apoyado en todo momento: gracias a ellos soy quien soy actualmente. También quiero dar las gracias a mis amigos: gracias a ellos he sido capaz de desconectar de vez en cuando; a mi tutor, Fernando Rincón, por su ayuda incluso estando de vacaciones, y al profesor David Mallasén, que ha tenido infinita paciencia para responder las preguntas que le he hecho por correo electrónico.

¡Muchas gracias a todos!

A mi familia, por estar ahí cuando más se necesita

Resumen

Este trabajo presenta el desarrollo de una infraestructura *hardware* para la integración de aceleradores de propósito específico en plataformas *RISC-V* utilizando la abstracción de tareas de FreeRTOS. La propuesta busca ofrecer al programador un modelo uniforme y eficiente, donde cada acelerador se trate como una tarea *hardware* que se ejecuta concurrentemente con las tareas *software* tradicionales. Para ello, se implementa una arquitectura basada en colas *hardware*, que facilitan la comunicación y sincronización entre el microcontrolador y los aceleradores, ocultando la complejidad de bajo nivel. La infraestructura se despliega sobre una extensión de la plataforma *X-HEEP*, denominada *GR-HEEP*, permitiendo validar su funcionalidad y explorar un marco de integración más generalizado. El enfoque presentado facilita la programación de sistemas embebidos heterogéneos, optimizando el uso de recursos especializados y promoviendo la portabilidad y accesibilidad del desarrollo en aplicaciones críticas de tiempo real.

Abstract

This work presents the development of a *hardware* infrastructure for the integration of application-specific accelerators on *RISC-V* platforms using the task abstraction provided by FreeRTOS. The proposed approach aims to offer programmers a uniform and efficient model, where each accelerator is treated as a *hardware* task that executes concurrently with traditional *software* tasks. To achieve this, an architecture based on *hardware* queues is implemented, facilitating communication and synchronization between the microcontroller and the accelerators while hiding low-level complexity. The infrastructure is deployed on the *X-HEEP* platform extension known as *GR-HEEP*, allowing validation of its functionality and exploration of a more generalized integration framework. The presented approach simplifies programming of heterogeneous embedded systems, optimizes the use of specialized resources, and promotes portability and accessibility in the development of real-time critical applications.

Acrónimos

ACAP Adaptive Compute Acceleration Platforms

ARM Advanced RISC Machines

ASIC Application Specific Integrated Circuit

AXI Advanced eXtensible Interface

DSP Digital Signal Processor

DVCS Distributed Version Control System

DUT Device Under Test

EDA Electronic Design Automation

EPFL Escuela Politécnica Federal de Lausanne

FIFO First In First Out

FFT Fourier Fast Transform

FPGA Field Programmable Gate Array

FSM Finite State Machine

GCC GNU Compiler Collection

GR-HEEP Generic Rich HEAP

HDL Hardware Description Languages

HLS High-Level Synthesis

IA Inteligencia Artificial

IDE Integrated Development Environment

IEEE Institute of Electrical and Electronics Engineers

IoT Internet of Things

IP Intellectual Property

ISA Instruction Set Architecture

MS Microsoft

RISC-V Reduced Instruction Set Computing V

RTL Register Transfer Level

RTOS Real Time Operating Systems

OBI Open Bus Interface

PUD Procesos Unificados de Desarrollo

SoC System-on-Chip

TFM Trabajo Fin de Máster

UUT Unit Under Test

UVM Universal Verification Methodology

VHDL Very High Speed Integrated Circuit Hardware Description Language

X-AIF eXtensible Accelerator InterFace

X-HEEP eXtensible Heterogeneous Energy-Efficient Platform

Índice general

1. Introducción	1
2. Estado del arte	5
2.1. Conceptos Fundamentales	5
2.1.1. RISC-V	5
2.1.2. FPGA	6
2.1.3. Diseño <i>hardware</i>	7
2.1.4. Herramientas utilizadas	17
2.1.5. Sistemas Operativos de Tiempo Real (RTOS)	22
3. Diseño de la solución	25
3.1. Metodología seguida	25
3.1.1. Procesos Unificados de Desarrollo	25
3.1.2. Iteraciones realizadas	26
3.2. Análisis de objetivos	27
3.3. Arquitectura de la solución	28
3.3.1. Componentes funcionales	28
3.3.2. Interfaces utilizadas	28

3.3.3. Componentes técnicos	29
3.3.4. Implementación e infraestructura	30
4. Resultados	31
4.1. Iteración 1 - Modelo del sistema	31
4.2. Iteración 2 - Desarrollo del diseño hardware	32
4.2.1. Diseño inicial	32
4.2.2. Diseño final	33
4.3. Iteración 3 - Ejecución de simulaciones funcionales	49
4.3.1. Testbench desarrollado	49
4.4. Iteración 4 - Integración del diseño en GR-HEEP	53
4.4.1. Importación de los archivos del diseño	54
4.4.2. Conexionado físico del diseño	56
4.4.3. Definición de los registros expuestos	57
4.4.4. Desarrollo de un <i>driver</i> para los registros	61
4.4.5. Compilación completa del sistema	64
4.5. Iteración 5 - Ejecución del diseño en GR-HEEP sobre FreeRTOS	64
4.5.1. Ejecución <i>baremetal</i>	65
4.5.2. Ejecución sobre FreeRTOS	68
4.6. Iteración 6 - Comparativa de resultados frente a versión software	73
5. Conclusiones y trabajo futuro	75
5.1. Competencias adquiridas	75
5.2. Revisión de los objetivos	76
5.3. Valoración crítica del proyecto	77

5.3.1. Líneas de trabajo futuro	77
5.3.2. Valoración personal	77
A. Anexo A	79
A.1. Código RTL de la primera versión del diseño	79

Índice de figuras

1.1. Logo de RISC-V. Wikimedia Commons	1
1.2. Logo de X-HEEP. Fuente: [EPF25a]	3
2.1. El primer chip RISC-V europeo, EPAC. Fuente: [Xat21]	6
2.2. Diligent Nexys A7. Fuente: [AMD22]	7
2.3. Block Design del diseño realizado en Vivado	14
2.4. Handshake básico del protocolo AXI. Fuente: [Jen22]	15
2.5. Arquitectura de X-HEEP, sobre la que se basa GR-HEEP. Fuente: [EE24]	18
2.6. Proyecto abierto	20
2.7. Proyecto abierto	21
2.8. Logo de GTKWave	22
2.9. Logo de FreeRTOS. Fuente: [Com25]	24
3.1. Diagrama de Gantt	27
4.1. Digrama de bloques en Vivado	38
4.2. Simulación de Vivado vista con su visor integrado	51
4.3. Simulación de EdaPlayground vista con GTKWave	52
4.4. Arquitectura de GR-HEEP integrando un acelerador (5) Fuente: [Qui25a]	53

4.5. Diagrama de conexión entre X-HEEP y el acelerador a través de las FIFOs y buses	54
4.6. Importando diseño en la plataforma	55
4.7. Concepto del diseño a integrar en la plataforma	56
4.8. Resultado de la ejecución <i>baremetal</i>	68
4.9. Resultado de la ejecución sobre FreeRTOS	72

Capítulo 1

Introducción

«If we knew what we were doing, it wouldn't be called research, would it?»

Albert Einstein

La arquitectura Reduced Instruction Set Computing V (RISC-V) se ha consolidado en los últimos años como una de las propuestas más relevantes dentro del ámbito de los sistemas embebidos y de propósito general. Su carácter *abierto, modular y extensible*, al contrario que alternativas tradicionales como Advanced RISC Machines (ARM) ha favorecido la creación de un ecosistema creciente de desarrolladores e investigadores interesados en explorar nuevas técnicas de optimización y personalización del *hardware*. En particular, el hecho de poder añadir extensiones específicas y adaptarlas a aplicaciones concretas convierte a *RISC-V* en una plataforma atractiva para la integración de aceleradores de propósito específico, cuyo objetivo principal es mejorar el rendimiento y la eficiencia energética en tareas críticas.



Figura 1.1: Logo de RISC-V. Wikimedia Commons

Sin embargo, uno de los principales desafíos asociados al uso de aceleradores en plataformas embebidas es la complejidad de su gestión desde el *software*. Normalmente,

los programadores deben interactuar con registros, controladores de interrupción y protocolos de comunicación de bajo nivel, lo que incrementa la dificultad del desarrollo y limita la portabilidad de las aplicaciones. En este sentido, se vuelve necesario diseñar mecanismos de abstracción que oculten la complejidad técnica del *hardware* y permitan a los desarrolladores aprovechar los aceleradores de manera sencilla y directa.

Dentro de este marco, los Real Time Operating Systems (RTOS) se presentan como un punto de apoyo fundamental. Estos proporcionan un entorno de ejecución que organiza, planifica y sincroniza tareas, garantizando un comportamiento determinista en aplicaciones críticas. Entre ellos, FreeRTOS se ha consolidado como una de las opciones más utilizadas debido a su simplicidad, escalabilidad y amplia adopción en la industria y la academia. FreeRTOS ofrece un conjunto de primitivas, como tareas, colas, semáforos y temporizadores, que facilitan la construcción de aplicaciones concurrentes, además de constituir una interfaz familiar para los programadores. En el presente trabajo propone el desarrollo de una infraestructura *hardware* orientada a la integración de aceleradores en plataformas *RISC-V* mediante el modelo de tareas de FreeRTOS. Para lograr este propósito, se plantea una arquitectura basada en colas *hardware*, que actúan como punto de comunicación entre el *software* que se ejecuta en el microcontrolador y los aceleradores de propósito específico. Estas colas permiten enviar y recibir datos, así como gestionar el estado de ejecución de los aceleradores, con un grado de generalidad suficiente para dar soporte a distintos tipos de unidades funcionales. En la práctica, el programador podrá interactuar con estas colas mediante las primitivas ya conocidas de FreeRTOS, sin necesidad de incurrir en configuraciones adicionales ni en dependencias con detalles de bajo nivel. El objetivo es, facilitar y unificar la experiencia de programación, abstrayendo el *hardware* del *software*.

Debido al elevado coste de fabricar prototipos físicos, la simulación se convierte en un paso imprescindible en el ciclo de desarrollo de *hardware*. A través de plataformas de ciclo preciso o de alto nivel, es posible validar el comportamiento funcional, analizar el rendimiento y detectar problemas de temporización sin necesidad de reconfigurar constantemente el FPGA. Proyectos como la plataforma eXtensible Heterogeneous Energy-Efficient Platform (X-HEEP) permiten experimentar con configuraciones de núcleos variadas, periféricos personalizados y sistemas completos, reduciendo riesgos y acelerando la depuración y optimización del diseño. Será una extensión de *X-HEEP*, llamada Generic Rich HEEP (GR-HEEP), la que se empleará como base para la implementación y validación de la infraestructura propuesta, gracias a que ofrece un entorno flexible y modular, facilitando la integración, así como la capacidad de simular arquitecturas *RISC-V* con recursos de memoria, periféricos y soporte para extensiones, lo que lo convierte en un candidato idóneo para desplegar arquitecturas experimentales de este tipo. Además, su carácter de plataforma *open-source* favorece la reproducibilidad de los resultados y la posibilidad de extender el trabajo hacia nuevas direcciones en el entorno investigador. En 1.2 se presenta el logo de X-HEEP.

El objetivo de esta investigación será, por tanto, demostrar la viabilidad y las

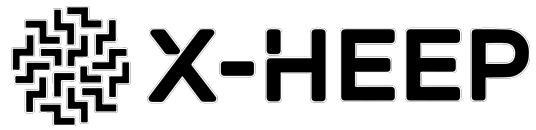


Figura 1.2: Logo de X-HEEP. Fuente: [EPF25a]

potenciales ventajas de una infraestructura *hardware* que permita integrar aceleradores de propósito específico como tareas en FreeRTOS, abstrayendo el *hardware* del *software*. De esta forma, se pretende avanzar hacia un modelo en el que los sistemas embebidos con procesadores *RISC-V* y aceleradores heterogéneos puedan ser programados de manera más accesible.

Capítulo 2

Estado del arte

En el siguiente capítulo de este Trabajo Fin de Máster (TFM) se presentan los conceptos y herramientas que es necesario comprender para poder seguir correctamente el flujo de este trabajo.

2.1. Conceptos Fundamentales

2.1.1. RISC-V

RISC-V es un Instruction Set Architecture (ISA) abierta y libre de *royalties*, desarrollada originalmente en la Universidad de Berkeley. Desde sus inicios en 2010, ha sido diseñada para permitir implementaciones ligeras, de bajo consumo y fácilmente adaptables para diversos dispositivos. Además, su flexibilidad, bajo costo y capacidad de personalización lo posicionan como un serio competidor frente a arquitecturas más tradicionales como ARM, con el principal objetivo de desbancarlo en términos de rendimiento y consumo, siendo precisamente éstos los más importantes para el proyecto actualmente. Si bien es posible, gracias a la capacidad de modularización, estar presente en diferentes escenarios, RISC-V tiene su mayor relevancia en sistemas embebidos y sensores Internet of Things (IoT), estimándose que miles de millones de unidades equipadas con RISC-V ya están en circulación [Int21].

RISC-V potencia la colaboración e innovación al ser una plataforma abierta que no requiere revelar propiedad intelectual, lo cual permitiría a, por ejemplo, Europa, a diseñar sus propios procesadores sin depender de terceros, siendo prueba de ello la fuerte inversión que está realizando en potenciar esta industria [Eur24]. De hecho, fruto de esta inversión es precisamente este programa de máster, enmarcado en el programa PERTE-CHIP [SER22]. Un ejemplo de procesador RISC-V se presenta en 2.1.



Figura 2.1: El primer chip RISC-V europeo, EPAC. Fuente: [Xat21]

2.1.2. FPGA

Los Field Programmable Gate Array (FPGA) surgieron en la década de 1980 como una solución intermedia entre circuitos integrados de propósito general y los Application Specific Integrated Circuit (ASIC), chips de propósito específico, con el objetivo de tener una plataforma con capacidad suficiente para desplegar prototipos *hardware* y agilizar los procesos de desarrollo. Fue precisamente la empresa Xilinx, actualmente AMD, la que introdujo en 1985 el primer FPGA comercial, permitiendo a los diseñadores configurar y reconfigurar la lógica interna del chip un número casi ilimitado de veces, perfecto para pruebas [Rom19]. En la actualidad, los FPGAs han pasado de ser componentes para lógica reconfigurable a Adaptive Compute Acceleration Platforms (ACAP)s [AMD25d], plataformas heterogéneas de aceleración que integran bloques de CPU, motores para Inteligencia Artificial (IA) y Digital Signal Processor (DSP), memoria de alta velocidad y conectividad coherente. Además, la adopción en la nube y el empuje por herramientas (propietarias y open-source), los ponen como opción clave para aceleración en centros de datos y computación en el *edge*. En este contexto, destaca Vivado Design Suite [AMD25c], la herramienta de desarrollo empleada en este trabajo.

Los retos actuales incluyen la reducción de la complejidad del flujo de diseño, el desarrollo de estándares abiertos para operar entre plataformas de fabricantes distintos, y el equilibrio entre programabilidad y rendimiento. A pesar de que el uso de lenguajes de alto nivel gracias a la adopción del High-Level Synthesis (HLS) ha simplificado el desarrollo, todavía existe una curva de aprendizaje pronunciada. En el futuro, se espera que los FPGA integren cada vez más interfaces coherentes como CXL, se adapten mejor a los flujos de trabajo de IA y expandan su papel en computación en el *edge* y aplicaciones de carácter científico. En la figura 2.2 puede verse la placa FPGA Nexys A7, de Diligent, empleada a lo largo del máster.

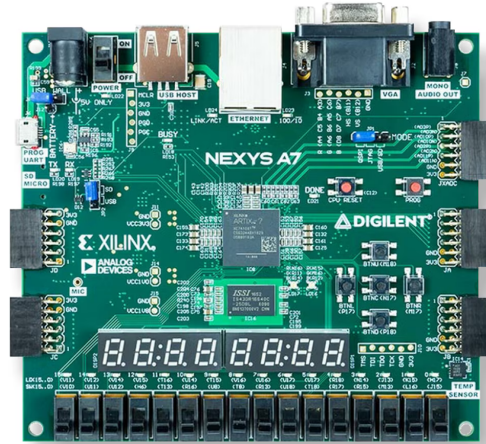


Figura 2.2: Diligent Nexys A7. Fuente: [AMD22]

2.1.3. Diseño *hardware*

Es el proceso de concebir, describir y realizar un sistema directamente en términos de sus componentes físicos y su comportamiento electrónico, sin abstraerse totalmente al software o a un nivel puramente algorítmico [Jim23], lo que implica definir cómo estará estructurado y cómo funcionará el circuito que implemente la funcionalidad deseada.

Diferencias con el diseño *software*

A diferencia del diseño *software*, el diseño *hardware* no trata de escribir un programa que un procesador interpretará, sino de definir el propio procesador y/o el circuito que ejecutará una tarea planteada. Una comparativa puede verse en 2.1.

Tabla 2.1: Comparación entre diseño a nivel de hardware y diseño a nivel de software

Aspecto	Diseño a nivel de hardware	Diseño a nivel de software
Descripción	Definición de la funcionalidad de un sistema mediante componentes físicos y lógica digital.	Definición de instrucciones y algoritmos que serán ejecutados por un procesador o máquina virtual.
Lenguajes	HDL (Verilog, SystemVerilog) y herramientas de síntesis/implementación.	Lenguajes de alto o medio nivel (C, C++, Python, Java, etc.).
Unidad de trabajo	Señales, registros, puertas lógicas, bloques IP, rutas de datos, temporización.	Variables, estructuras de datos, funciones, hilos, procesos.
Ejecución	El comportamiento está “fijo” en el hardware (aunque pueda ser reconfigurable en FPGA).	Se ejecuta secuencial o concurrentemente sobre hardware ya existente.
Paralelismo	Intrínseco: múltiples operaciones ocurren en paralelo por diseño físico.	Generalmente limitado al paralelismo gestionado por hilos/procesos del software.
Tiempo de desarrollo	Mayor complejidad y validación más costosa, incluye simulación, síntesis, implementación y pruebas en hardware real.	Más rápido de iterar y depurar; la validación se hace en un entorno controlado (IDE, simuladores de software).
Optimización	Área, consumo de energía, latencia y frecuencia de operación.	Tiempo de ejecución, uso de memoria, escalabilidad del código.

Principales puntos de vista

Los principales puntos de vista del diseño a nivel *hardware* son los siguientes:

- **Diseño lógico:** descripción del comportamiento con HDLs como Verilog, especificando registros, puertas lógicas, Finite State Machine (FSM)s, y protocolos de interconexión a utilizar.
- **Diseño estructural:** organización y conexionado de los módulos o IPs del diseño, integrándolos en FPGAs, ASICs o dispositivos similares mediante el uso de líneas de comunicación y/o protocolos de interconexión planteados en el diseño lógico.
- **Diseño físico:** es la fase del diseño donde se implementa el diseño de forma física en el silicio, incluyendo ubicación de los módulos/IPs, enrutado de las conexiones

y optimizaciones de área y consumo.

- **Diseño eléctrico:** definición de niveles de voltaje y limitaciones de corriente máxima dentro del circuito, comprobaciones temporales de las señales y su estabilidad, así como compatibilidad con los buses/protocolos de interconexión utilizados.

Conceptos clave

Metodología EDA

A grandes rasgos, pueden verse dos tipos principales de metodologías: las de gestión de proyectos, y las técnicas, centradas en la optimización del producto final. Será precisamente en ésta última categoría donde entran las metodologías del diseño de *hardware*. En este contexto, se entiende por *metodología técnica* al conjunto de enfoques, procedimientos y herramientas utilizadas para diseñar, desarrollar, verificar y optimizar un producto desde el punto de vista técnico o ingenieril. [Sch02]. En este contexto, se emplean metodologías variadas, como *Top-Down*, que descompone el sistema en módulos jerárquicos hasta llegar a los módulos básicos, y *Bottom-Up*, que consiste en construir el sistema a partir de módulos básicos hasta llegar a abarcar completamente el sistema a desarrollar.

Sin embargo, la metodología relevante dentro de este proyecto es la basada en herramientas Electronic Design Automation (EDA), siendo la metodología base para las herramientas utilizadas en este trabajo, como es la Vivado Design Suite, ya mencionada. Esta metodología adopta un enfoque estructurado e iterativo que se extiende desde la definición de requisitos hasta la fabricación física. El flujo de la metodología EDA se basa en dos grandes bloques: Front-End y Back-End, con verificación continua en todas las etapas. [Rin24]. Este flujo se detalla en 2.2.

Gracias a esta metodología, el diseño de ASICs y sistemas complejos integra fases iterativas, reutilización de IPs, modelado jerárquico y verificación exhaustiva, buscando optimizar coste, tiempo y fiabilidad.

Front-end y Back-end

El Front-End se refiere a la fase inicial de planteamiento del diseño *hardware*, centrada en la funcionalidad y la lógica del circuito, antes de realizar implementación física alguna. Por otra parte, el Back-end va después del Front-end, y se centra en la implementación física del diseño planteado y en preparar el circuito para la fabricación en silicio [Rin24]. En 2.3 se presentan detalladamente las principales actividades de estas dos fases.

Etapa	Sub-etapas
Especificación y planificación	Requisitos funcionales y no funcionales; Arquitectura y selección de IP cores; Modelado de alto nivel: SystemC, Matlab
Front-End	Diseño algorítmico y HLS; Diseño lógico RTL: Verilog, SystemVerilog; Reutilización de IP (Soft y Hard); Síntesis lógica a netlist
Back-End	Planificación física, placement y routing; Síntesis y distribución de reloj; Verificación física: DRC y LVS
Verificación	Funcional y formal; Análisis de tiempo estático; Simulación post-layout
Sign-off y fabricación	Validaciones finales y análisis de consumo; Generación de vectores de test (DFT, BIST); Creación de archivos GDSII y tape-out

Tabla 2.2: Resumen de las actividades en cada etapa del flujo de la metodología EDA

Hardware Description Language (HDL)

Los Hardware Description Languages (HDL)s son lenguajes de programación especializados utilizados para describir, modelar y simular el comportamiento y la estructura de circuitos electrónicos digitales. A diferencia de los lenguajes de programación tradicionales (como C o Python), los HDLs permiten especificar tanto el comportamiento lógico como la implementación física de un sistema, posibilitando su síntesis en dispositivos de hardware como FPGAs o ASICs. Su aparición en la segunda mitad del siglo XX revolucionó la forma en que ingenieros y diseñadores desarrollan hardware, permitiendo ciclos de diseño más rápidos y complejos. [Wik23]. Entre los HDLs más conocidos se encuentran Verilog y Very High Speed Integrated Circuit Hardware Description Language (VHDL). De todos ellos, el más representativo en el contexto de este trabajo es Verilog, el cual será detallado en posteriores apartados.

Hardware Level Synthesis (HLS)

HLS, también conocido como *behavioral synthesis* o *algorithmic synthesis*, es un proceso automatizado que transforma una descripción funcional de alto nivel, como escrita en C, C++, SystemC, MATLAB o similar en una representación de hardware en HDLs como Verilog [Eng22]

A la hora de diseñar *hardware* para probar en FPGAs, HLS ofrece ventajas como un desarrollo más rápido, una mayor abstracción y portabilidad entre plataformas. Permite a los diseñadores trabajar con lenguajes de alto nivel como C o C++, algo especialmente interesante para diseñadores de *software*. La depuración del código generado por HLS puede ser un desafío y

Etapa	Descripción
Front-End	
Diseño algorítmico	Modelado funcional de los bloques o sistema completo sin preocuparse por la implementación física (ej. algoritmos en SystemC o Matlab).
Síntesis de alto nivel (HLS)	Traducción de modelos funcionales a RTL considerando restricciones de frecuencia, área y consumo.
Diseño lógico RTL	Descripción detallada de la lógica interna, rutas de datos y control usando Verilog o SystemVerilog, lista para síntesis.
Reutilización de IP	Integración de bloques ya diseñados: Soft IP (modificable, en RTL) o Hard IP (cerca de silicio, optimizado).
Síntesis lógica	Conversión final de RTL a netlist de puertas lógicas adaptada a la tecnología seleccionada.
Back-End	
Planificación física	Distribución de bloques y componentes en el chip para optimizar área y conexiones.
Placement	Ubicación óptima de celdas para minimizar retardos y conexiones.
Routing	Trazado de interconexiones físicas de celdas en el chip o configuración de recursos en FPGA.
Síntesis y distribución de reloj	Creación de la red de sincronización para que todos los bloques funcionen coordinadamente.
Verificación física	Comprobación de reglas de diseño (DRC) y equivalencia entre esquemático y layout (LVS).

Tabla 2.3: Resumen de etapas del Front-End y Back-End en diseño digital

la calidad de salida de la herramienta depende de la experiencia del diseñador, el cual, para alcanzar una mayor optimización del diseño generado, deberá revisar la generación obtenida. [Tos22]

Las señales Clock y Reset

En procesadores, microcontroladores o circuitos implementados en FPGA, las señales Clock y Reset son fundamentales para que el hardware funcione de manera ordenada y predecible.

La señal de *Clock* (reloj) es una onda periódica, normalmente cuadrática, que marca el ritmo al que un circuito digital ejecuta operaciones. [WH15] [HH21]. Su principal función es sincronizar el funcionamiento de todos los módulos donde esté conectada, como flip-flops, registros y contadores, de manera que los cambios de estado ocurran únicamente en momentos definidos, normal-

mente en los bordes de subida (*active-high*) o bajada (*active-low*) de la señal. Asimismo, el término *frecuencia de clock*, se refiere a la velocidad con la que se producen estas subidas y bajadas de la señal, siendo a mayor frecuencia, mayor el número de transiciones por unidad de tiempo, lo que desemboca en una mayor capacidad de procesamiento, mientras que su ciclo de trabajo y estabilidad en las transiciones de subida y bajada influyen en la fiabilidad de las operaciones de los módulos conectados a dicha señal.

Por otro lado, La señal de *Reset* tiene la función de inicializar el sistema a un estado conocido y seguro [Cor21]. Al activarse, fuerza a todos los registros y elementos secuenciales a tomar valores iniciales, garantizando que el sistema comience siempre en condiciones predecibles. Existen dos tipos principales: el reset síncrono, que actúa junto con el clock, y el reset asíncrono, que se activa inmediatamente sin esperar un ciclo de reloj. Esta señal es crucial al encender un dispositivo, para evitar estados indeterminados, y también se utiliza para reiniciar el sistema en caso de errores o bloqueos.

La importancia conjunta de ambas señales es clara: el *clock* determina cuándo ocurren las acciones dentro del hardware, mientras que el *reset* define desde dónde y en qué condiciones se empieza a trabajar. Sin un clock estable, el sistema pierde el orden en sus operaciones; sin un reset confiable, el arranque puede ser inestable, por tanto, poco seguro, lo que podría desembocar en graves accidentes allá donde esté desplegado el circuito en cuestión.

Módulo RTL y módulo IP

Un módulo Register Transfer Level (RTL) es un bloque de hardware descrito en un lenguaje de descripción de hardware como Verilog, del cual se hablará posteriormente, y donde se especifica el comportamiento y la estructura del circuito a nivel de registros, señales y operaciones lógicas. El diseñador puede escribir este código manualmente o generarlo con herramientas automatizadas, y normalmente se tiene acceso al código fuente, lo que permite modificarlo, optimizarlo o adaptarlo a distintas plataformas. Un módulo RTL describe con detalle cómo funciona internamente el hardware y es portable. Por tanto, el módulo RTL se centra en el diseño detallado y editable del hardware. [ISD23]

Por otro lado, un módulo Intellectual Property (IP) es un bloque de hardware ya diseñado, probado y empaquetado para ser reutilizado en múltiples proyectos. Generalmente es proporcionado por un fabricante o un tercero (por ejemplo, Xilinx o Intel/Altera) y puede venir como un diseño cerrado (*black-box*) o con opciones limitadas de configuración. [Awa22]. Su uso suele ser más rápido porque evita diseñar el hardware desde cero, pero muchas veces no se puede ver ni modificar su implementación interna, ya que forma parte de la propiedad intelectual del proveedor. Por tanto, el módulo IP es una unidad

reutilizable, e inmutable, que puede ser integrada en un diseño para agregarle funcionalidades.

La diferencia principal entre ambos es que el módulo RTL es un diseño abierto y editable que describe el funcionamiento interno del hardware en detalle, de tipo *white-box*, mientras que un módulo IP es un bloque ya terminado y optimizado que normalmente se usa como componente prefabricado, a menudo sin acceso a su código interno, pues, por norma general, está protegido por licencias.

Módulo Top

El módulo top (conocido también por *top-level module*) es la unidad más elevada en la jerarquía de la arquitectura del sistema *hardware* diseñado. Este módulo no es instanciado dentro de otro, y sirve como contenedor de todos los submódulos necesarios para materializar el diseño completo, actuando como el punto de integración final antes de su implementación física [Chi22]. Es en el módulo Top donde se define y organiza la interfaz global con el exterior, como los pines I/O de un FPGA, a la vez que sincroniza la interconexión entre bloques internos que tiene instanciados.

Wrapper

Por otra parte, el *wrapper* desempeña un rol igualmente crucial como intermediario entre un módulo RTL o IP ya existente con el resto del sistema. En plataformas como Vivado Design Suite, herramienta empleada en este trabajo, el *wrapper* se genera para conectar los puertos de entrada y salida del diseño con los pines físicos definidos en el archivo de restricciones. Asimismo, el *wrapper* también facilita la adaptación entre jerarquías, ajustes en nombres de señales o protocolos, y la integración de módulos que de otra forma serían incompatibles [For06]. El uso de *wrappers* permite mantener la integridad del módulo original (por ejemplo, un IP de terceros), mientras se ajusta a requerimientos externos sin modificar su código fuente. El objetivo general de un *wrapper* es, en múltiples ocasiones, facilitar la conectividad exclusivamente, lo que hace que casi siempre estén carentes de lógica específica.

Llegados a este punto, puede parecer que el módulo Top y un *Wrapper* son muy parecidos. En realidad, desempeñan funciones complementarias. El primero constituye la instancia principal que integra el diseño, mientras que el segundo actúa como un mediador que permite integrar módulos de cualquier tipo, como IPs de terceros, los cuales, en numerosos casos, presentan especificaciones técnicas distintas respecto al resto del diseño. [For06].

Block Design

En el diseño de *hardware* digital, el concepto de *Block Design* se refiere a la construcción de sistemas a partir de bloques modulares que poseen funcionalidades concretas [Xil23]. Cada bloque representa un módulo, que puede ser desde un procesador, un controlador de memoria, o un módulo personalizado. Estos bloques se conectan entre sí a través de interfaces, como el bus Advanced eXtensible Interface (AXI).

Una de las principales ventajas del enfoque de Block Design es la modularidad, permitiéndose el reuso de IPs de terceros, así como módulos personalizados, dentro de un mismo desarrollo. De esta manera, no se necesita trabajar directamente a nivel de puertas lógicas o escribir RTL desde cero, basta con importar cada componente dentro del diseño y generar un *wrapper* que contenga información de todos los presentes dentro del mismo. Así, se acelera el proceso de desarrollo y mejora la escalabilidad del diseño, permitiendo a la vez una visualización clara y jerárquica del sistema, útil para entender y depurar arquitecturas complejas [Xil23] [MK04].

En la práctica, este método es ampliamente utilizado en el desarrollo *hardware*. En el contexto de este trabajo, se ha realizado el desarrollo del diseño a nivel *hardware* precisamente sobre un Block Design, por la cómoda integración y facilidad de uso dentro de Vivado Design Suite. En la figura 2.3 se presenta un ejemplo de Block Design realizado con Vivado Design Suite

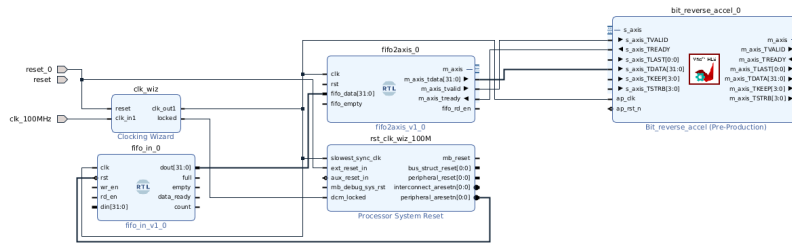


Figura 2.3: Block Design del diseño realizado en Vivado

Testbench

En el diseño de circuitos digitales, un *testbench* o banco de pruebas es un archivo escrito generalmente en lenguajes HDL como Verilog, y su función principal es aplicar estímulos al circuito bajo prueba, el Device Under Test (DUT), (o el Unit Under Test (UUT), si se quiere probar un módulo y no el conjunto completo), para finalmente observar las respuestas que se producen, y así comprobar el cumplimiento de los requisitos. El testbench no es un módulo más del diseño; es una herramienta de simulación. Dentro de él, se generan señales de entrada, se definen patrones de prueba (como impulsos de reloj, valores de datos, resets, etc.) y se supervisan las salidas del DUT por medio de equivalencias con lo esperado y aserciones.

Existen distintos niveles de sofisticación en los testbenchs: desde versiones simples que solo aplican algunos vectores de entrada manualmente, hasta entornos más complejos con generación automática de estímulos, verificación dirigida por cobertura (coverage-driven verification), e incluso entornos basados en metodologías formales como UVM (Universal Verification Methodology). En el máster se ha trabajado con UVM, sin embargo, los testbenchs realizados no lo emplean principalmente por errores incomprensibles al intentar integrarlo en el testbench utilizado para las simulaciones en Vivado.

AXI Stream

El protocolo AXI Stream es una interfaz estándar de un solo sentido utilizada para la transmisión de datos de forma continua y eficiente entre componentes conectados, sin necesidad de direcciones o complejas fases de control [ARM23]. Constituye un enlace punto a punto entre un transmisor (*master*) y un receptor (*slave*), basado en una simple señalización de *handshake* mediante un par de señales:

- TVALID: indica que el transmisor está enviando datos válidos.
- TREADY: indica que el receptor está listo para recibir datos.

Cuando se cumple la condición de que ambas señales están en alto en el mismo ciclo, se realiza la transferencia de datos por TDATA [Jen22]. Para determinar si el dato a enviar es el último o no se tiene la señal TLAST, también empleada en este proyecto para implementar correctamente el flujo de los datos. Hay más señales, como TKEEP, TSTRB, TID, TDEST o TUSER, que permiten delimitar paquetes, identificar flujos o transmitir datos auxiliares [AMD25e], aunque estas últimas no tendrán relevancia en este proyecto.

El esquema básico del *handshake* se presenta en 2.4.

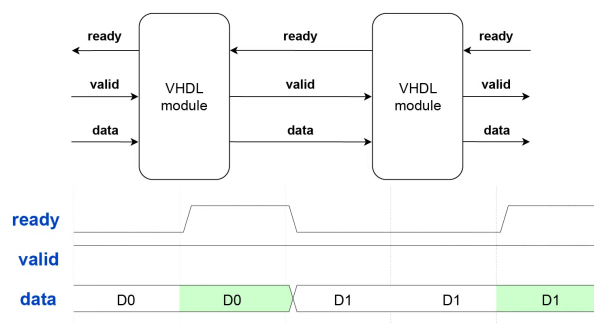


Figura 2.4: Handshake básico del protocolo AXI. Fuente: [Jen22]

OBI

Open Bus Interface (OBI) es un protocolo abierto, como su propio nombre indica, empleado en la arquitectura RISC-V, especialmente dentro del proyecto CORE-V del grupo OpenHW, para interconexiones punto a punto entre núcleos como el CV32E40* y módulos de memoria o periféricos [Gro21] [Gro23].

OBI se distingue por su diseño simple y eficiente, enfocado en facilitar la interconexión entre IPs abiertos dentro del ecosistema RISC-V, sin la complejidad de protocolos más pesados, lo que favorece la implementación modular y la interoperabilidad. En el contexto de este trabajo, se trabajará tanto con los protocolos AXI Stream, para el acelerador, y OBI, para comunicar el diseño desarrollado con la plataforma completa creada con X-HEEP.

Las diferencias entre AXI-Stream y OBI se presentan en 2.4

Característica	AXI-Stream	OBI (Open Bus Interface)
Tipo de comunicación	Flujo de datos continuo, unidireccional, sin direccionamiento.	Transacciones direccionadas (lectura/escritura) hacia memoria y periféricos.
Uso principal	Transmisión de datos en tiempo real: vídeo, señales digitales, comunicaciones.	Interconexión de CPU RISC-V con memoria y periféricos.
Señales clave	TVALID, TREADY, TDATA, opcionalmente TLAST, TKEEP, etc.	Señales de dirección, datos de lectura/escritura, tamaño de acceso (byte, media palabra, palabra).
Complejidad	Simple, diseñado solo para transferencias de datos.	Ligero, pero más completo: soporta direccionamiento y control.
Flexibilidad	Altamente eficiente para flujos secuenciales y de gran ancho de banda.	Diseñado para ser modular, interoperable y escalable en sistemas RISC-V.
Ejemplo de aplicación	Pipeline de datos en procesadores de señal digital (DSP) o bloques de vídeo.	Interfaz Harvard en procesadores CORE-V (separación de instrucciones y datos).

Tabla 2.4: Comparación entre AXI-Stream y OBI

2.1.4. Herramientas utilizadas

X-HEEP y su extensión GR-HEEP

X-HEEP [MSM⁺24] [EE24] es una arquitectura de microprocesador de tipo RISC-V y de carácter *open-source*, desarrollada en SystemVerilog por el Escuela Politécnica Federal de Lausanne (EPFL). Su diseño modular permite personalización de *cores*, periféricos, memorias y topologías para habilitar aceleradores especializados sin modificar la plataforma base. X-HEEP puede implementarse sobre distintas tecnologías, incluyendo FPGAs, siendo ideal para este trabajo enfocado en el prototipado de microcontroladores RISC-V.

En este proyecto se utilizará una extensión llamada GR-HEEP [Tor24], una extensión del System-on-Chip (SoC) base de X-HEEP. Concretamente, se empleará del repositorio de David Mallasén [Qui25b]. GR-HEEP ofrece una plataforma más completa y preconfigurada, facilitando el uso de periféricos y aceleradores externos, sin embargo, actualmente está todavía en fase de prueba. Las principales diferencias entre X-HEEP y GR-HEEP se muestran en la tabla 2.5.

Aspecto	X-HEEP	GR-HEEP
Naturaleza	Plataforma base minimalista	Extensión preconfigurada de X-HEEP
Desarrollador	EPFL ESL Lab	VISI Lab, Universidad de Bolonia
Propósito	Exploración de aceleradores ultra-low-power	Prototipado rápido con periféricos adicionales
Periféricos	Básicos	Amplia gama (UART, I2C, SPI, etc.)
Configuración	Mediante <code>mcu-gen</code> y JSON	Hereda de X-HEEP con configuraciones adicionales

Tabla 2.5: Comparación entre X-HEEP y GR-HEEP

GR-HEEP mantiene la organización modular por dominios de energía (CPU, periféricos, memoria, *always-on*) y utiliza cores RISC-V de 32 bits estilo Harvard, sin caché, adecuados para aplicaciones de bajo consumo y RTOS integrados [EE25]. La figura 2.5 muestra la arquitectura de X-HEEP, sobre la cual se basa GR-HEEP. En este proyecto, se usará específicamente para prototipado en FPGA.

Verilog

Creado en 1984 por Phil Moorby y Prabhu Goel en Gateway Design Automation, su diseño buscaba una sintaxis familiar a programadores en C, lo que contri-

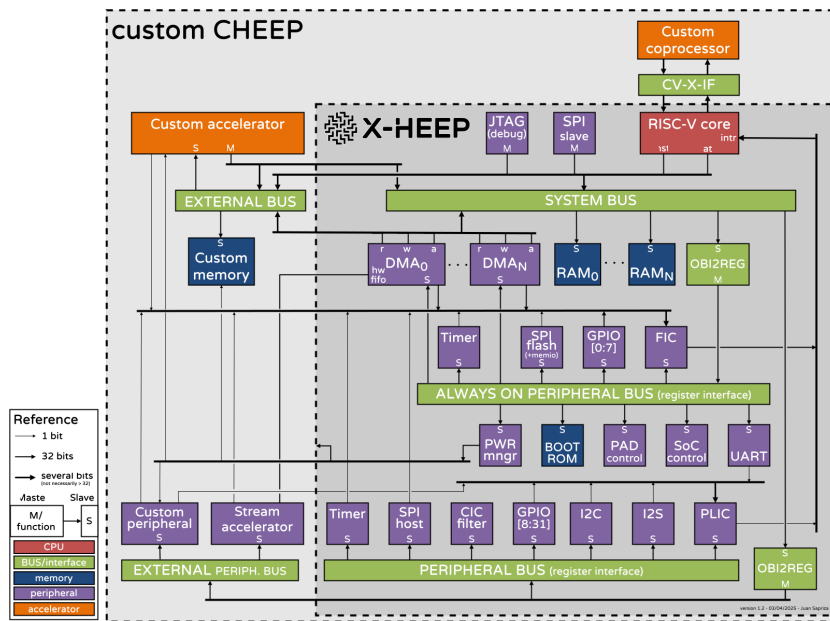


Figura 2.5: Arquitectura de X-HEEP, sobre la que se basa GR-HEEP. Fuente: [EE24]

buyó a su rápida adopción. Fue estandarizado por primera vez como IEEE 1364-1995 (“Verilog-95”), seguido por revisiones en 2001 y 2005. En 2009, Verilog fue fusionado dentro de SystemVerilog bajo el estándar IEEE 1800-2009 y actualmente forma parte del estándar unificado IEEE 1800-2023. [IEE01] Verilog ocupa un lugar central en el diseño digital moderno debido a su equilibrio entre legibilidad, flexibilidad y potencia. Fue originalmente un lenguaje propietario en los años 80, diseñado para modelado de hardware, y en la década de los 90 se abrió al dominio público a través de Open Verilog International (OVI), que lo llevó a estandarización del Institute of Electrical and Electronics Engineers (IEEE) [Rea20]. Su adopción global y la abundancia de herramientas de simulación y síntesis compatibles lo han consolidado como uno de los pilares del diseño digital moderno. En 2.1 puede verse un ejemplo de un sumador desarrollado en Verilog.

```

1  module adder_4bit(
2      input    [3:0] a,
3      input    [3:0] b,
4      input          cin,
5      output [3:0] sum,
6      output          cout
7  );
8
9      // Salidas del sumador
10     assign {cout, sum} = a + b + cin;
11

```

```
12 endmodule
```

Vivado Design Suite

Vivado Design Suite [AMD25c], desarrollado originalmente por Xilinx en 2012, y ahora parte de AMD, es una plataforma integrada en la metodología basada en herramientas EDA. Sustituta del antiguo ISE Design Suite, fue creada desde cero para abordar las crecientes exigencias de diseño en FPGAs y SoCs modernos [Mic24].

Vivado está construido sobre un modelo de datos compartido y escalable, que permite una integración fluida entre diseño, síntesis, análisis, simulación, depuración y cierre de tiempos (*timing closure*). Su arquitectura favorece la visibilidad continua de métricas clave como utilización de recursos, potencia, congestión o tiempos, en todas las etapas del flujo de diseño. [Mic14]. Será, por tanto, Vivado Design Suite la empleada para la realización del diseño *hardware* desarrollado en este trabajo.

Las principales herramientas presentes en Vivado Design Suite son:

- **Vivado IDE:** Interfaz gráfica principal para capturar el diseño, gestionar proyectos, correr síntesis, implementación y generar el bitstream. Incluye consola Tcl para automatización y scripting.
- **Vivado Synthesis:** Motor de síntesis lógica para VHDL, Verilog y SystemVerilog, que convierte el código HDL en una netlist optimizada para la FPGA.
- **Vivado Implementation:** Realiza place & route, optimiza para timing closure y genera el bitstream final del diseño.
- **Vivado Simulator (XSim):** Simulador HDL integrado y multilenguaje para depurar el diseño antes de implementarlo en hardware.
- **Vivado IP Integrator:** Entorno gráfico para ensamblar IPs predefinidos y crear subsistemas usando interfaces como AXI de forma automática.
- **Vivado High-Level Synthesis (HLS):** Herramienta para diseñar en C, C++ o SystemC y generar hardware a partir de código de alto nivel.
- **Vivado Logic Analyzer:** Permite la depuración en hardware mediante analizadores lógicos integrados (ILA) y monitorización de señales internas en tiempo real.
- **Vivado Power Analysis:** Estimación y análisis de consumo de potencia del diseño en distintas fases del flujo.

- **Vivado ECO Editor:** Permite aplicar Engineering Change Orders al diseño implementado sin necesidad de recompilar todo.

Para finalizar, se presenta en 2.6 un proyecto abierto en Vivado Design Suite.

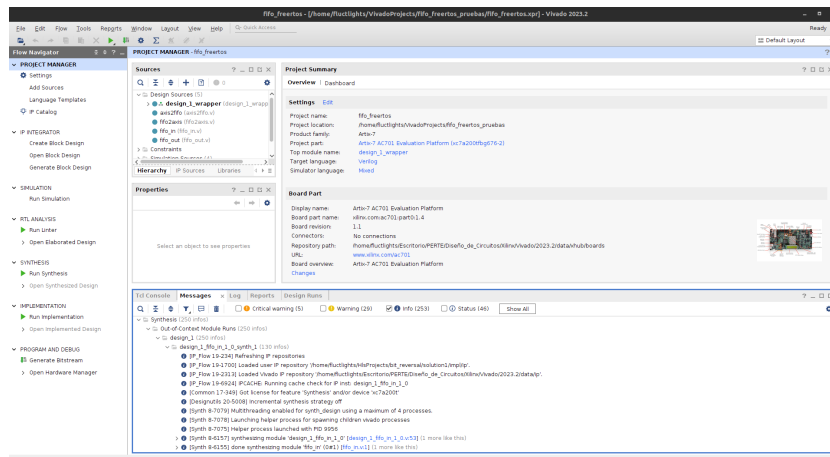


Figura 2.6: Proyecto abierto

Vitis HLS

Vitis HLS es una herramienta de síntesis de alto nivel desarrollada por Xilinx (ahora propiedad de AMD) que permite la conversión de código en C/C++ a RTL sintetizable para dispositivos FPGAs [AMD25a]. Esta herramienta facilita la creación de algoritmos complejos para *hardware* sin necesidad de escribir código a lenguajes de descripción de *hardware* tradicionales como VHDL o Verilog. Una de las principales ventajas de Vitis HLS es su capacidad para abstraer el diseño de hardware, permitiendo a los diseñadores centrarse en la funcionalidad del algoritmo en lugar de los detalles de implementación del hardware. La herramienta soporta directivas de síntesis que permiten optimizar aspectos como la paralelización, la unroll de bucles y la partición de arrays, lo que resulta en una implementación más eficiente en términos de recursos y rendimiento. Además, Vitis HLS ofrece simulación en C para validar la funcionalidad del diseño antes de la síntesis, lo que acelera el ciclo de desarrollo y mejora la productividad [AMD25b].

Vitis HLS está estrechamente integrado con Vivado Design Suite [Sha23], permitiendo exportar el diseño sintetizado como un bloque IP, que puede ser importado directamente en Vivado para su integración en un diseño más amplio. Este flujo de trabajo facilita la creación de sistemas complejos que combinan lógica personalizada generada por HLS con bloques IP preexistentes. De esta forma, permite una transición fluida desde el desarrollo de algoritmos hasta los primeros pasos para la implementación en *hardware*.

Nuevamente, para terminar este apartado, se presenta en 2.7 un proyecto abierto en Vitis HLS.

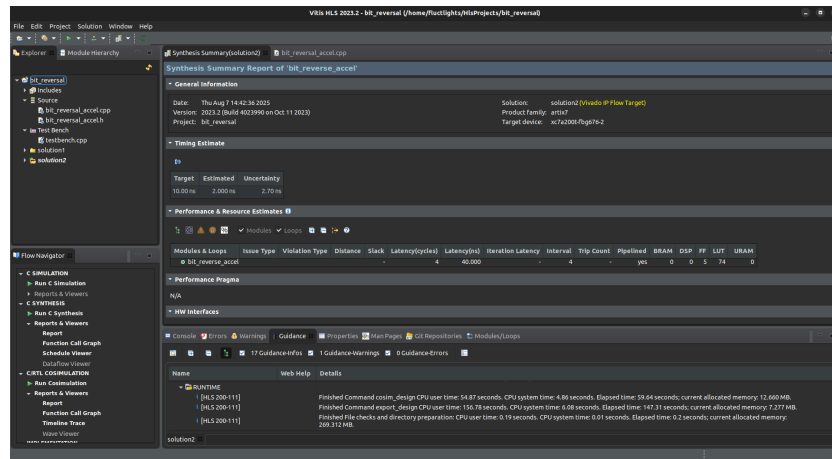


Figura 2.7: Proyecto abierto

EdaPlayground

EDAPlayground es una plataforma en línea gratuita que permite escribir, simular y compartir código de diseño digital utilizando lenguajes de descripción de hardware como Verilog o SystemVerilog. Esta herramienta está orientada a estudiantes e ingenieros que quieren realizar simulaciones y compilaciones de diseños hardware sin tener que instalar software especializado. EdaPlayground tiene además, la posibilidad de compartir proyectos fácilmente mediante enlaces, lo convirtiéndolo en una herramienta ideal para tareas de equipo y docencia. Es precisamente por estas cualidades que ha sido la herramienta elegida como alternativa a la Vivado Suite para realizar pruebas de simulación, pudiendo verse el Playground en cuestión en el siguiente enlace.

La plataforma ofrece un Integrated Development Environment (IDE) web, donde los usuarios pueden crear módulos de diseño y *testbenchs*, además de compilarlos y simularlos utilizando diferentes herramientas de simulación proporcionadas por proveedores como Synopsys. Entre los simuladores disponibles se encuentran ModelSim, o el utilizado en el contexto de este trabajo, QuestaSim, de Siemens, en su versión 2023.2. Además, EdaPlayground permite visualizar resultados de simulación, ya sea en forma de texto EPWave, lo que facilita el análisis del comportamiento del circuito. Sin embargo, debido a la complejidad de la simulación, se ha dependido de otra herramienta de visualización llamada *GTKwave*, explicada posteriormente.

GTKwave

GTKwave es un software de visualización de señales de onda, al igual que EPWave de EdaPlayground. Éste es, sin embargo, mucho más potente, pues permite ver un mayor intervalo de tiempo, entre otras cuestiones y al estar instalado en local (es una instalación muy ligera y fácil de instalar con Aptitude) posee una mayor integración con el entorno gráfico del ordenador en cuestión. Para los diagramas de onda de EdaPlayground se ha utilizado GTKWave por su gran ligereza y potencia, así como su fácil usabilidad.



Figura 2.8: Logo de GTKWave

2.1.5. Sistemas Operativos de Tiempo Real (RTOS)

Un RTOS es un tipo especializado de sistema operativo diseñado para gestionar tareas que deben ejecutarse dentro de límites de tiempo estrictos y predecibles. A diferencia de los sistemas operativos de propósito general, como Windows, que están orientados a la multitarea y la gestión eficiente de recursos, un RTOS se centra en garantizar que las tareas críticas se completen dentro de plazos específicos [Tec24] [Win23]. Si bien hay multitud de opciones a día de hoy, destacan, entre otros, Zephyr [Pro25] y FreeRTOS [Fre25], siendo este último el utilizado en este TFM.

FreeRTOS

Originalmente desarrollado por Richard Barry en 2003, y mantenido por Amazon desde 2017, FreeRTOS es un *kernel* de sistema operativo de tiempo real de código

abierto, diseñado específicamente para sistemas embebidos con recursos limitados. Además, recientemente ha recibido soporte para RISC-V [Auf19], lo que lo hace idóneo para este proyecto. Las características principales de FreeRTOS son:

- **Pequeño y eficiente:** diseñado para tener una huella de memoria mínima, lo que lo hace adecuado para microcontroladores con recursos limitados.
- **Multiplataforma:** compatible con más de 40 arquitecturas, incluyendo ARM, AVR, RISC-V, entre otras.
- **Planificación preemptiva:** implementa un planificador de tareas que permite la ejecución de tareas con prioridades, garantizando que las tareas críticas se ejecuten a tiempo.
- **Soporte para multitarea:** permite la crear y gestionar múltiples tareas, semáforos, colas y temporizadores, facilitando la programación concurrente.
- **Licencia MIT:** es de código abierto, lo que permite su uso y modificación sin restricciones.

En el contexto de este trabajo, FreeRTOS se utilizará para gestionar de manera eficiente la ejecución de tareas software y hardware en la plataforma *RISC-V* GR-HEEP. Cada tarea hardware se planificará de la misma forma que una software de FreeRTOS, lo que permitirá una abstracción entre el software desarrollado y la infraestructura. Esto se logrará mediante el uso de colas First In First Out (FIFO). Una cola FIFO es una estructura de datos en la que el primer elemento que entra es el primero que sale, siguiendo el principio de una fila en la vida real. En hardware, las colas FIFO suelen implementarse como bloques de memoria con dos punteros (lectura y escritura) y lógica de control que garantiza que los datos se lean en el mismo orden en que fueron escritos. En el diseño digital, las FIFO se usan como *buffers* entre módulos que operan a velocidades o relojes diferentes, o para absorber variaciones en el flujo de datos. Pueden ser síncronas (un solo dominio de reloj) o asíncronas (diferentes dominios de reloj en lectura y escritura, con circuitos de sincronización). En el contexto de este trabajo, las FIFO utilizadas son de carácter síncrono, a modo de simplificar el diseño final.

Las colas FIFO presentan un considerable número de ventajas, destacando:

1. Aislamiento entre productores y consumidores
2. Simplificación de la lógica de control
3. Facilidad para integrarse en arquitecturas *pipeline*
4. Versatilidad en IP cores

5. Evitan pérdidas de datos, sirviendo de almacenamiento temporal

El objetivo principal será, por tanto, gestionar la comunicación y coordinación entre el microcontrolador y los aceleradores, de forma que se garantice la integridad de los datos y el cumplimiento de restricciones de tiempo real, siendo al mismo tiempo una forma sencilla de entrada y salida de datos. De esta forma, el programador podrá interactuar con los aceleradores como si de una tarea software se tratara, utilizando la misma interfaz y ocultando la complejidad del hardware, ofreciendo un modelo homogéneo, escalable y fácilmente extensible para futuras implementaciones. Para finalizar, se presenta en 2.9 el logo.



Figura 2.9: Logo de FreeRTOS. Fuente: [Com25]

Capítulo 3

Diseño de la solución

En este capítulo se expondrán la metodología seguida en el trabajo, los objetivos del mismo, y presentación de la solución desarrollada.

3.1. Metodología seguida

Se define metodología [Esp], como los métodos y pasos seguidos con el objetivo de desarrollar o fabricar algo, de manera que el resultado final sea acorde a los resultados y calidad esperados del mismo. De esta forma, es posible cumplir con los plazos de entrega del producto o servicio, a la vez que se obtiene un resultado acorde a lo establecido.

3.1.1. Procesos Unificados de Desarrollo

En la actualidad, es posible encontrar innumerables metodologías para gestionar un proyecto de cualquier tipo. Si bien es imposible o muy difícil adecuarse por completo a una sola metodología, es posible decantarse por un conjunto reducido o incluso una sola en función del objetivo del proyecto a desarrollar. En este trabajo se ha emplear una metodología basada en Procesos Unificados de Desarrollo (PUD) [Jac00] [AF15], centrada en el enfoque iterativo e incremental con el objetivo de aportar un mayor valor conforme avanza el proyecto a realizar. Como principales razones para esta elección destacan las siguientes:

- El proyecto se organiza de manera estructurada en iteraciones
- Cada iteración puede verse como un micro-proyecto, permitiendo, si es necesario, la aplicación de metodologías distintas, dando gran flexibilidad.

- Los resultados de cada iteración sirven como base para la siguiente, aumentando progresivamente el valor del proyecto a medida que éste avanza.
- No se necesita definir todos los requisitos al comienzo, es posible realizar ajustes en iteraciones previas sin dificultad.
- Agiliza el desarrollo del proyecto al generar resultados a corto plazo

En la metodología PUD, el esfuerzo se divide en fases, cada una tomando unas entradas, y generando unas salidas, las cuales serán la entrada de la siguiente fase a realizar. El conjunto de todas las fases es lo que se conoce como ciclo. Así pues, un ciclo está compuesto de las siguientes fases:

1. **Inicio:** Se describe del resultado esperado al fin de la iteración.
2. **Elaboración:** Se detallan los casos de uso que a considerar durante la iteración.
3. **Construcción:** Se modela el resultado de la iteración a partir de los casos de uso. En caso de fallo de los requisitos, se realimenta la fase hasta obtener un resultado correcto.
4. **Transición:** Se valida el resultado generado en la iteración. En caso de fallo de los requisitos, se realimenta la fase hasta obtener un resultado correcto.

En este trabajo este ciclo se repetirá las veces necesarias para cumplir con los objetivos planteados en el mismo.

3.1.2. Iteraciones realizadas

A modo de esquema, se presenta en 3.1 una tabla relacionando las iteraciones realizadas en este proyecto, fechas de inicio/fin, descripciones, dentro del único ciclo de PUD realizado. Para finalizar, se presenta en 3.1 el diagrama de Gantt equivalente.

FASE	ITERACIÓN	DESCRIPCIÓN	INICIO	FINAL
Inicio	1	Modelo del sistema	31/05/2025	15/06/2025
Elaboracion	2	Desarrollo del diseño hardware	10/06/2025	31/07/2025
	3	Ejecución de simulaciones funcionales	01/08/2025	10/08/2025
Construccion	4	Integración del diseño en X-HEEP	11/08/2025	25/08/2025
	5	Ejecución del diseño en X-HEEP sobre FreeRTOS	16/08/2025	01/09/2025
Transicion	6	Comparativa de resultados frente a versión software	30/08/2025	03/09/2025
	7	Documentación del proyecto	01/08/2025	05/09/2025

Tabla 3.1: Iteraciones identificadas en el proyecto

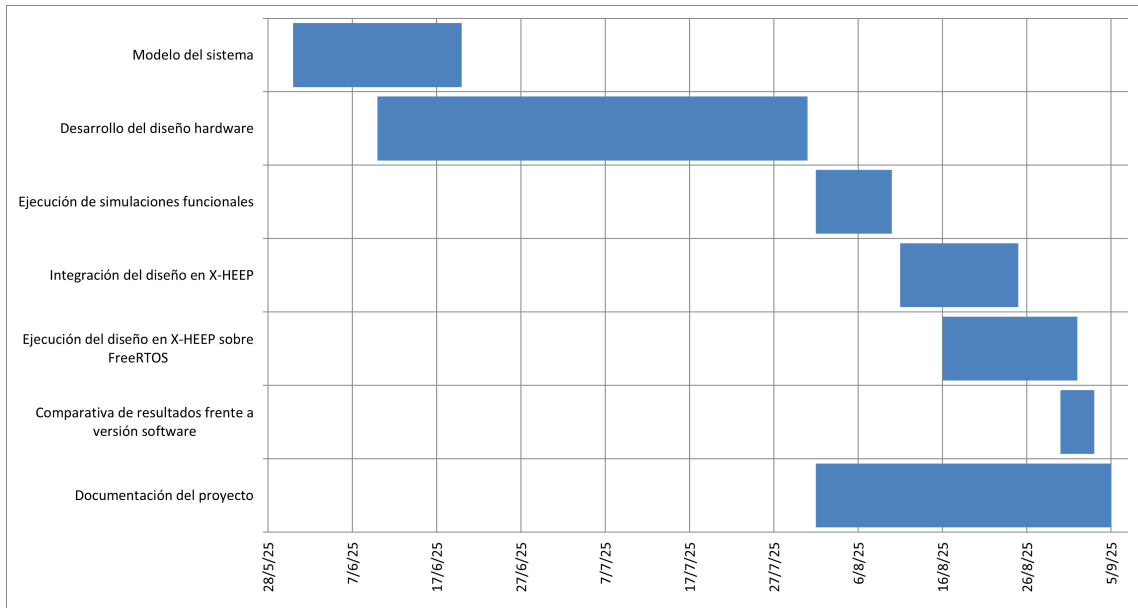


Figura 3.1: Diagrama de Gantt

3.2. Análisis de objetivos

El objetivo principal del proyecto será **la implementación e integración en GR-HEEP de una tarea hardware sobre el sistema operativo de tiempo real FreeRTOS**, de modo que sea más eficiente que una realizada en software. Para el cumplimiento de este objetivo, se tienen una serie de objetivos parciales:

1. *La implementación de un mecanismo basado en colas FIFO*, que sirvan de nexo entre el *software* ejecutado en el microcontrolador y el hardware.
2. *La utilización del protocolo AXI-Stream para la entrada y salida de datos del acelerador* empleado en la tarea hardware a desarrollar.
3. *La validación de la infraestructura hardware mediante simulaciones y ejecuciones*, que permita verificar la funcionalidad, empleado para ello herramientas de simulación y de ejecución que permitan obtener datos acerca del correcto funcionamiento del mismo.
4. *La integración de la infraestructura hardware desarrollada en GR-HEEP*, para poder implementar y ejecutar la tarea hardware.
5. *La adición del RTOS FreeRTOS a GR-HEEP y al software ejecutado en el microcontrolador*, para poder ejecutar las tareas desarrolladas.
6. *El desarrollo de drivers que permitan la comunicación hardware-software*, y así servir de mediador en la comunicación entre el diseño realizado y los programas software empleando el sistema operativo de tiempo real FreeRTOS

7. La comparativa de rendimiento entre la tarea *hardware* y una tarea *software* del mismo tipo, como forma de evaluación real relativa al *speedup* logrado con la tarea *hardware* a desarrollar.

El cumplimiento (completo o parcial) o incumplimiento de estos objetivos se valorará con posterioridad, en el capítulo 5.

3.3. Arquitectura de la solución

3.3.1. Componentes funcionales

En el proyecto se ha llevado a cabo una solución basada en diagramas de bloques (*block design*) de carácter modular, donde cada bloque presenta una funcionalidad específica, y su agrupación permite realizar funciones de mayor complejidad. Concretamente, se han realizado los siguientes bloques:

1. Una cola de entrada de datos, que permite la entrada de enteros de 32 bits, así como una señal de comienzo (*start*) que arranca el diseño globalmente.
2. Una versión acelerada por *hardware* del algoritmo de reverso de bits (*bit reversal*) realizada con el programa Vitis HLS, e importado a Vivado
3. Una cola de salida de datos, que permite obtener el resultado final tras aplicar el reverso, así como una señal de terminado (*done*) para saber en qué momento se tienen resultados.

Estos elementos se han integrado en un Block Design en Vivado, que es el que instancia a todos ellos, tal y como puede verse en 2.6.

3.3.2. Interfaces utilizadas

Para la interconexión de los módulos de las colas con el acelerador en cuestión, se ha hecho uso del conocido protocolo AXI Stream, donde se ha realizado una implementación reducida, teniendo en cuenta, por simplicidad, el menor número de señales posible: *datos*, *ready*, *valid*, *last*. En la figura 2.6 puede apreciarse estas conexiones citadas.

Además de AXI, es necesario utilizar otro protocolo para poder integrar el diseño en X-HEEP: OBI, protocolo libre de licencias y utilizado en los SoCs RISC-V,

RI5CY y LowRISC-Ibex *cores* [Gro21], como el CV32E40P. Este protocolo es similar a AXI, y se basa igualmente en un esquema *request-response*.

3.3.3. Componentes técnicos

Si bien las herramientas principales, empleadas para el diseño de la solución, ya han sido detalladas:

- **Vivado Design Suite:** en 2.1.4
- **X-HEEP/GR-HEEP:** en 2.1.4
- **FreeRTOS:** en 2.1.5
- **EdaPlayground:** en 2.1.4

Se debe, como mínimo, presentar una serie de herramientas extra, clave para el despliegue técnico del proyecto, destacando:

- **Vitis HLS:** [AMD25a] herramienta que convierte un algoritmo en C a código HDL como Verilog, clave para poder integrarlo con otros módulos en *hardware*.
- **Make**[Sta99]: herramienta para compilar la plataforma de X-HEEP
- **RISC-V GCC-Toolchain**[Fou25]: para compilar programas RISC-V con GNU Compiler Collection (GCC)
- **Visual Studio Code**[Mic25]: como el IDE empleado para el desarrollo técnico del proyecto

Hay que presentar también herramientas utilizadas en la gestión del proyecto a nivel global:

- **Git:** es un Distributed Version Control System (DVCS), sistema de control de versiones distribuido, que permite mantener un historial del código desarrollado en cada momento, pudiendo volver atrás si es necesario. En Git, un proyecto está compuesto por un *repositorio*, o un conjunto de éstos.
- **GitHub:** plataforma donde los usuarios pueden alojar repositorios Git y registrar los cambios realizados. Para este trabajo, se ha creado un repositorio, al cual puede accederse en este enlace

- **ClickUp**: herramienta web de gestión de proyectos. Permite conocer el estado en todo momento y a planificar diagramas de Gantt cómodamente, entre otros.

Y finalmente, las herramientas utilizadas en la redacción de este proyecto son:

- **L^AT_EX**: sistema de procesamiento de documentos basado en TeX, muy popular en el ámbito académico, principalmente por la alta calidad de los documentos finales, lo que facilita la lectura.
- **Latex Workshop**: extensión para Visual Studio Code que proporciona soporte a características y compilación para documentos L^AT_EX.
- **Drawio**: editor web que sirve, entre otros, para haber realizado los diagramas presentes en el trabajo, con gran facilidad y flexibilidad de uso.
- **Tables Generator**: editor web empleado para realizar las tablas presentes en el trabajo.
- **Microsoft (MS) Excel**: gestor de hojas de cálculo polivalente, empleado en este trabajo para ordenar datos y generar las tablas comparativas de rendimiento entre diseño desarrollado y versión puramente software.

3.3.4. Implementación e infraestructura

La implementación del proyecto se ha realizado en tres fases bien diferenciadas:

1. **Desarrollo del algoritmo *bit reversal***: mediante la herramienta Vitis HLS, se ha generado el código RTL necesario para realizar la implementación en *hardware*.
2. **Creación del diseño basado en *block design***: mediante Vivado Design Suite se ha realizado y simulado con *testbenches* un diseño basado en colas que utiliza AXI-Stream para comunicarse con el acelerador desarrollado.
3. **Integración del diseño en X-HEEP sobre FreeRTOS**: se hace uso de un *fork* de X-HEEP llamado GR-HEEP [Tor24] para ejecutar software en C con las librerías de FreeRTOS y haciendo uso del diseño creado. Será precisamente esta integración la intraestructura final sobre la que se sostenga el trabajo.

Capítulo 4

Resultados

En el siguiente capítulo se detallarán los desarrollos y se presentarán los resultados. Para una mejor comprensión, se dividirá en diversas secciones, una por cada una de las iteraciones realizadas en este trabajo.

4.1. Iteración 1 - Modelo del sistema

Esta iteración se centra en la descripción del problema planteado y en la elección de las herramientas a utilizar para el desarrollo de un diseño *hardware* capaz de cumplir con los objetivos expuestos en 3.2. En este punto, se ha analizado en detalle el conjunto de dichos objetivos, además de realizar diversas preguntas, para entender mejor la problemática.

Con el objetivo de cumplir dichos objetivos, se pretende desarrollar un diseño *hardware* capaz de realizar una operación concreta, siendo, en el contexto de este trabajo, el reverso de bits de un número entero de 32 bits. Para ello, se ha optado por un esquema basado en colas FIFO. La idea es, por tanto, hacer sencillo el paso de datos al acelerador, así como la recepción de los resultados. Las colas FIFO, al tener memoria interna, proporcionan un mecanismo sencillo y relativamente seguro de transmisión.

Para realizar este diseño, es necesario realizarlo en base a una metodología. En el contexto de este trabajo, se ha optado por alinear el esfuerzo a las metodologías basadas en herramientas EDA, ello implicando el uso de programas software que estén adecuados al flujo de este tipo de metodologías. Por ello, se ha optado por seleccionar Vitis HLS y Vivado como programas para el diseño de este hardware. Ambas son herramientas de la misma compañía, Xilinx, lo que repercute en, teóricamente, una mejor integración entre ellas, lo que agiliza el desarrollo. Además, han sido presentadas en el máster, lo que las hace más fáciles de utilizar que herramientas de la competencia, como las de la empresa Cadence. Para pruebas, ya con un SoC simulado, se ha optado por el uso de X-HEEP, que permite la creación de núcleos RISC-V de múltiples tipos, así

como simulaciones a nivel *hardware* gracias a Verilator, y con soporte para FreeRTOS, lo que proporciona flexibilidad a la hora de realizar cualquier tipo de despliegue.

Por tanto, a modo de resumen, el resultado de esta iteración ha sido:

1. Se han analizado los objetivos
2. Se ha proporcionado un marco de trabajo para el desarrollo técnico del proyecto
3. Se ha dado respuesta a cómo comunicarse a la entrada y a la salida del módulo acelerador
4. Se han seleccionado e instalado las herramientas a utilizar tras valorar diversas opciones

4.2. Iteración 2 - Desarrollo del diseño hardware

En esta fase se ha realizado el esquema correspondiente al un diseño hardware que sea capaz de satisfacer los objetivos y cumplir con el problema planteado. Si bien se estudiaron varios mecanismos, tal y como se expone en 4.1, se ha optado por un esquema basado en colas FIFO.

4.2.1. Diseño inicial

Para lograr el diseño final, han sido necesario un refinamiento del planteamiento inicial. Originalmente, se ideó un esquema de 4 módulos más el acelerador por separado. El código de éstos se presenta en A.1, a excepción del acelerador, pues este no cambió. Los módulos desarrollados en esta primera versión fueron los siguientes:

1. **Cola FIFO de entrada:** cuyo objetivo es guardar datos para, llegados a un *threshold*, comenzar a mandarlos al siguiente. Mediante una señal de escritura, se escriben los datos. A partir de entonces, es posible leerlos por la salida.
2. **Cola FIFO de conversión a AXI-Stream:** este módulo se comunica con el acelerador con señales AXI-Stream de entrada. Recibe los datos guardados por el módulo anterior, y, recibidos todos (ese *threshold*) el módulo activa las señales AXI-Stream para mandar los datos al módulo acelerador
3. **Acelerador generado con Vitis HLS:** este módulo se basa en el protocolo AXI-Stream, de forma que, al activarse las señales de su entrada, comienza a recibir los datos y a procesarlos uno a uno hacia el siguiente módulo y activa señales de salida.

4. **Cola FIFO de conversión AXI al formato de origen:** este módulo se comunica con el acelerador con señales AXI-Stream de salida. Recibe los resultados y, al activarse sus entradas AXI-Stream, comienza a mandarlos al siguiente.
5. **Cola FIFO de salida:** este módulo obtiene los datos resultado del módulo conversos de AXI-Stream a datos puros. Una vez recibidos todos, los devuelve por su salida de datos.

Este esquema no ha sido el definitivo, entre otras, por las siguientes razones:

- **Demasiados componentes para la tarea a realizar:** se puede agruparlos sin aumentar en exceso la lógica de control del diseño global.
- **Inicialización del acelerador inconsistente:** no se tiene lógica alguna para activar el módulo HLS, activándose ésta al inicio, siendo necesario un tiempo extra para un correcto funcionamiento.
- **Falta de señales de control:** solo se devuelven los datos del acelerador, sin tener señales de control ni de estado que indiquen información acerca de la ejecución, lo que hace difícil la depuración en caso de fallo.
- **Presencia de código no sintetizable:** en el primer diseño se enfocó a pruebas de comportamiento en Vivado, no se realizó fase de síntesis, y en el código hay presente partes no sintetizables, algo problemático para las siguientes fases de desarrollo.

4.2.2. Diseño final

Descripción funcional

Finalmente se ha optimizado el diseño original, añadiendo señalizaciones de control y estado, y reduciendo a tres el número de módulos, que son:

1. **Cola FIFO de conversión de datos a AXI-Stream:** este módulo se comunica con el acelerador con señales AXI-Stream de entrada. Una vez que la señal de *reset* deja de estar activa, se activa la señal de control *start_accel*, que activa el módulo acelerador. Los datos fluyen de la siguiente forma:
 - Con la activación de la señal de control *write*, el módulo controla el guardado de datos, que están recibándose desde su señal de entrada *din*, en una variable interna llamada *buffer*, que contiene los datos a enviar al acelerador. Con todos los datos recibidos, llegados al *threshold*, en este contexto,

puesto para 4 valores, el módulo espera a que el acelerador esté dispuesto a recibir datos, algo que sucederá cuando la entrada de esta cola, llamada *s_axis_tready* se active.

- Ante este hecho, el módulo mete datos en la salida, llamada *s_axis_tdata*, y activa la salida *s_axis_tvalid*, señal que, una vez detectada por el acelerador, hace que éste mire su entrada de datos, y los procese.
 - Cuando todos los datos han sido enviados (se alcanzó el *threshold*) el módulo se espera a que se active su entrada *s_axis_tready*, marcando que el acelerador mandó todos los resultados. En este punto, se borra el contenido de salida de datos *s_axis_tdata* y el módulo se queda en estado de espera.
2. **Acelerador generado con Vitis HLS:** este módulo emplea en el protocolo AXI-Stream para comunicarse; ya inicializado gracias a la señal *start_accel* del módulo anterior, que pone a su señal *ap_rst_n* a 1 para comenzar. Una vez se comienza a los datos por su entrada *s_axis_TDATA* son validados con la señal *s_axis_TVALID*, comienza a procesarlos y los manda hacia el siguiente módulo metiendo los datos por su salida *m_axis_TDATA* y validando el resultado con *s_axis_TVALID*. Cuando se alcance el último elemento, se activa la señal *s_axis_TLAST*, para que los módulos de entrada y de salida de datos sepan que han llegado al final de la transmisión de datos.
3. **Cola FIFO de conversión AXI-Stream al formato de origen:** este módulo se comunica con el acelerador con señales AXI-Stream de salida. Los datos fluyen de la siguiente forma:
- Con la activación de su señal de control *valid_result*, el módulo controla el guardado de los datos recibidos por su otra señal *result_data* en una variable interna llamada *buffer*. Para recibir el dato, se activa la señal de salida *ready_for_results*, que indica al acelerador que puede enviar como tal los datos resultado en cuestión.
 - Cuando se activa la señal de entrada *last_result*, significa que se va a procesar el último resultado. En cuanto ésto sucede, se guarda como siempre en *buffer*, se vuelve a poner *ready_for_results* a 0, y se activa la señal de control *done*, que indica que los resultados ya están disponibles para leer.
 - Llegados a este punto, el módulo se mantiene a la espera de recibir una petición de lectura a través de su señal de entrada *read*. De esta forma, al activarse dicha señal, se mete el valor leído de *buffer* por la señal de salida *dout*.

Señales principales

Se presentan en 4.1 las señales más importantes del diseño hardware realizado en este trabajo.

Señal	Descripción
clk	Señal de reloj principal del sistema.
reset / rst	Señal de reset (activa en alto), reinicia el diseño.
din_i[31:0]	Entrada de datos de 32 bits hacia el sistema.
write_i	Permite escritura de un dato de 32 bits en la cola de entrada.
start_flag_i	Señal de inicio para habilitar el envío de datos.
fifo_data[31:0]	Datos provenientes del FIFO hacia el bus AXI-Stream.
s_axis_tdata[31:0]	Bus de datos de entrada (AXI-Stream).
s_axis_tvalid	Indica que los datos en <i>s_axis_tdata</i> son válidos.
s_axis_tready	Señal de ready"del esclavo, indica que puede aceptar datos.
s_axis_tlast	Indica la última transferencia de un paquete AXI-Stream.
m_axis_tdata[31:0]	Bus de datos de salida (AXI-Stream).
m_axis_tlast	Señal de fin de paquete de datos en la salida.
done_flag_o	Señal que indica finalización del procesamiento.
dout_o[31:0]	Datos de salida del sistema (32 bits).
read_i	Permite la lectura de un resultado de 32 bits en la cola de salida.
ap_clk	Reloj para el acelerador HLS.
ap_rst_n	Reset del acelerador HLS (activo en bajo).

Tabla 4.1: Principales señales del diseño

Módulo Top

Los módulos presentados son instanciados a través de un módulo adicional, que será el *módulo Top*, concepto explicado en 2.1.3. A partir de aquí, se puede agregar lógica adicional que no proceda integrar en otro lado. En este contexto, se ha insertado en el módulo Top la lógica destinada al correcto guardado y correcta lectura de los datos: esto es así porque, si bien en las simulaciones no se valora esta problemática, pues los cambios de las señales se realizan a nivel de ciclo. Sin embargo, en posteriores fases el diseño estará integrado en una plataforma como es GR-HEEP (X-HEEP), y las señales de control del diseño no se manejarán a nivel de ciclo, sino que habrá retardos, los cuales habrá que gestionar para asegurar el correcto funcionamiento del mismo.

Gracias al concepto de Block Design, generar un módulo Top es sencillo al menos, en la herramienta Vivado, con el IP Integrator. El código relativo al módulo Top se presenta en 4.1.

Listado 4.1: Código fuente del módulo Top

```

1 module bitreversal (
2     input [31:0] din_i,
3     output logic done_flag_o,
4     output logic [31:0] dout_o,
5     input logic start_flag_i,
6     input logic clk,
7     input logic read_i,
8     input logic write_i,
9     input logic rst_ni
10 );
11
12 reg internal_rst = 1; // reset signal for design_1

```

```
13  reg [1:0] rst_cnt    = 0;    // counter for reset delay
14
15  reg internal_read    = 0;    // read signal for design_1
16  reg already_read     = 0;    // check if reading has occurred
17  reg [1:0] read_cnt   = 0;    // counter for readings
18
19  reg internal_write    = 0;    // write signal for design_1
20  reg already_written   = 0;    // check if writing has occurred
21  reg [1:0] write_cnt   = 0;    // counter for writings
22
23  design_1 design_1_i
24    (.clk(clk),
25     .din_i(din_i),
26     .done_flag_o(done_flag_o),
27     .dout_o(dout_o),
28     .read_i(internal_read),
29     .reset(internal_rst),
30     .start_flag_i(start_flag_i),
31     .write_i(internal_write)
32    );
33
34  always_ff @(posedge clk) begin
35
36    if (internal_rst) begin
37      if (rst_cnt < 3) begin //3 cycles
38        rst_cnt = rst_cnt + 1;
39      end else begin
40        read_cnt = 0;
41        rst_cnt = 0;
42        internal_rst = 0;
43      end
44    end else if (done_flag_o) begin
45
46      if(!internal_read && read_i && !already_read) begin
47        internal_read = 1;
48
49      end else if (internal_read && read_i && !already_read)
50        begin
51          internal_read = 0;
52          already_read = 1;
53
54          if (read_cnt < 3) begin
55            read_cnt = read_cnt + 1;
56          end else begin
57            read_cnt = 0;
58            internal_rst = 1;
59          end
60        end
61      end
```

```

59     end else if (!internal_read && !read_i && already_read)
60         begin
61             already_read = 0;
62         end
63     end else if (write_i) begin
64         if(!internal_write && !already_written) begin
65             internal_write = 1;
66         end
67         end else if (internal_write && !already_written) begin
68             internal_write = 0;
69             already_written = 1;
70         end
71         if (write_cnt < 3) begin
72             write_cnt = write_cnt + 1;
73         end else begin
74             write_cnt = 0;
75         end
76     end
77
78     end else if (!write_i) begin
79         already_written = 0;
80     end
81
82 end
83 endmodule

```

El módulo Top posee una serie de entradas: *din_i* (32 bits), que es el puerto de entrada de datos; *start_flag_i*, la señal que inicia dicho procesamiento; *clk*, la señal de reloj del sistema; *read_i* y *write_i*, que son señales externas para controlar la lectura y la escritura de datos, respectivamente; y *rst_ni*, una señal de reset activa en bajo que no se utiliza directamente, y que se mantiene para facilitar la integración con GR-HEEP. Por otro lado, las salidas del módulo incluyen: *done_flag_o*, que indica cuando el procesamiento ha finalizado, y *dout_o* (32 bits), puerto de salida de los datos procesados.

Se instancia *design_1*, que corresponde al RTL generado por Vivado tras realizar el Block Design, con todas las señales mencionadas. Como puede verse, en los campos de *read*, *write*, y *reset*, no se utilizan directamente *read_i*, *write_i*, *rst_ni*. Esto es, porque se ha desarrollado la lógica de control de estas señales para dentro del diseño como tal, de forma que se puedan gestionar correctamente lecturas y escrituras, así como no permitir que la señal de reset se pueda activar o desactivar desde fuera, sino que sea un proceso interno. La lógica de control del sistema se ejecuta en cada flanco positivo del reloj, siguiendo una secuencia precisa para cada operación. Inicialmente, al activarse el *reset* interno, se mantiene la señal *internal_rst* en alto durante tres ciclos

de reloj, utilizando el contador *rst_cnt*; transcurrido este tiempo, se desactiva y se inicia el diseño.

Por otro lado, la lógica de procesamiento completado gestiona las operaciones de lectura cuando la señal *done_flag_o* está activa. En este estado, si la señal de lectura externa *read_i* se activa y los datos no han sido leídos previamente (*already_read* a 0), se habilita la señal de lectura interna *internal_read*. En el siguiente ciclo de reloj, esta señal interna se desactiva y se marca la operación como completada. Después de que se han realizado todas las lecturas (en este contexto, 4), se resetea. Las operaciones de escritura se gestionan de manera similar: cuando la señal *write_i* está activa, se habilita una escritura interna (*internal_write*) por un solo ciclo, así hasta que sucedan todas las escrituras posibles (en este contexto, 4). Finalmente, la lógica también se encarga de resetear los flags de estado (*already_read* y *already_written*) tan pronto como las señales externas correspondientes *read_i* y *write_i* se desactivan.

Finalmente, el Block Design en la herramienta Vivado, con cada módulo integrado se presenta en 4.1.

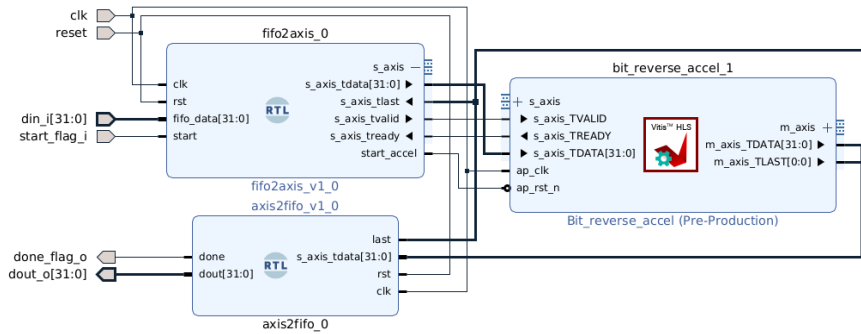


Figura 4.1: Digrama de bloques en Vivado

Código C/C++ del acelerador

El código relativo al acelerador del reverso de bits en C/C++ empleado para HLS se presenta en esta sección.

Listado 4.2: Código fuente en C/C++ del acelerador reverso de bits

```

1 #ifndef BIT_REVERSAL_ACCEL_H
2 #define BIT_REVERSAL_ACCEL_H
3
4 #include <hls_stream.h>
5 #include <ap_int.h>
6 #include <ap_axi_sdata.h>
7

```

```

8 // AXI Stream width: 32 bits of data, no side channels
9 typedef ap_axiu<32, 0, 0, 0> axis_t;
10
11 void bit_reverse_accel(hls::stream<axis_t>& s_axis,
12                       hls::stream<axis_t>& m_axis);
13
14 #endif
15 // Top function
16 void bit_reverse_accel(hls::stream<axis_t>& s_axis,
17                       hls::stream<axis_t>& m_axis) {
18
19 #pragma HLS INTERFACE axis port=s_axis
20 #pragma HLS INTERFACE axis port=m_axis
21 #pragma HLS INTERFACE ap_ctrl_none port=return
22 #pragma HLS PIPELINE II=3
23
24     axis_t input_word, output_word;
25     ap_uint<32> reversed;
26     int i,j = 0;
27
28     // Read 4 words
29     for (i = 0; i < 4; i++) {
30         input_word = s_axis.read();
31
32         for (j = 0; j < 32; j++) {
33 #pragma HLS UNROLL
34             reversed[31 - j] = input_word.data[j];
35         }
36
37         output_word.data = reversed;
38         output_word.last = (i == 3);           // signal last word
39                                                // in the stream
40         m_axis.write(output_word);
41     }

```

En 4.2 se presenta el `bit_reverse_accel`, que implementa un acelerador de hardware para realizar una inversión de bits (*bit-reversal*) sobre palabras de 32 bits, utilizando la interfaz AXI Stream. La función toma como entrada un flujo (*stream*) de datos de tipo `axis_t`, y produce un flujo de salida del mismo tipo. El tipo `axis_t` se define como una estructura AXI Stream de 32 bits (`ap_axiu<32,0,0,0>`), sin canales adicionales (como `keep`, `id`, `dest`, etc.).

La función es decorada con directivas específicas para Vitis HLS. En particular, las interfaces `s_axis` y `m_axis` se definen como puertos AXI4-Stream mediante

la directiva `#pragma HLS INTERFACE axis`, lo que permite que el acelerador se comunique directamente con otros módulos de hardware o con un procesador. También se usa `#pragma HLS INTERFACE ap_ctrl_none port=return`, lo cual indica que no hay control explícito de inicio o fin (es decir, no se utilizan señales como `ap_start` ni `ap_done`), y la función opera de forma continua. Además, se emplea la directiva `#pragma HLS PIPELINE II=3`, solicitando un *Initiation Interval (II)* de 3 ciclos de reloj, permitiendo así procesar un nuevo dato cada 3 ciclos. Si bien, podría hacerse en uno solo, a modo de prueba, se ha decidido dejarlo así.

Dentro del cuerpo de la función, se procesan 4 palabras de 32 bits. En cada iteración del bucle principal (`for (i = 0; i < 4; i++)`), se lee una palabra del stream de entrada, y se aplica un bucle anidado que recorre los 32 bits de la palabra, invirtiendo su orden. Esta operación se realiza bit a bit: el bit en la posición `j` del dato de entrada se coloca en la posición `31 - j` de la variable `reversed`, logrando así la inversión. Este bucle está completamente desenrollado con la directiva `#pragma HLS UNROLL`, permitiendo que todos los bits se procesen en paralelo, en un solo ciclo.

Una vez calculada la palabra invertida, se empaqueta en una nueva estructura `output_word`, incluyendo el campo `last`. Este campo indica si la palabra es la última del flujo, lo cual ocurre cuando `i == 3`. Finalmente, el dato transformado se escribe en el flujo de salida `m_axis`.

Código RTL desarrollado

El código relativo a cada uno de los módulos RTL se presenta en esta sección.

Listado 4.3: Código fuente de la cola FIFO conversora a AXI-Stream

```

1 module fifo2axis #(
2     parameter DATA_WIDTH = 32,
3     parameter THRESHOLD = 4,
4     parameter DEPTH = 4
5 ) (
6     input wire clk,
7     input wire rst,
8
9     // FIFO interface
10    input wire [31:0] din,
11    input wire        start,
12    input wire        write,
13
14    // AXI Stream master
15    output reg  [31:0] s_axis_tdata,
16    output reg          s_axis_tvalid,

```



```

17  input  wire      s_axis_tready,
18  output wire      start_accel,
19  input  wire      s_axis_tlast
20 );
21
22  // States
23  localparam IDLE = 3'd0, WRITE_FIFO = 3'd1, WAIT_START = 3'd2,
24             SEND = 3'd3, DONE = 3'd4;
25
26  reg [2:0] state, next_state;
27
28  // Internal buffer
29  reg [31:0] buffer          [0:3];
30  reg [ 1:0] buffer_index;
31  reg [ 1:0] send_index;
32
33  // When RESET signal is 0 and module starts, also start the
34  // HLS module
35  assign start_accel = ~rst;
36
37  // FSM Sequential
38  always @(posedge clk or posedge rst) begin
39      if (rst) begin
40          state = IDLE;
41          buffer_index <= 2'd0;
42          send_index <= 2'd0;
43          s_axis_tvalid <= 0;
44          s_axis_tdata <= {32{1'bx}};
45      end else begin
46          state = next_state;
47
48          case (state)
49              IDLE: begin
50                  s_axis_tvalid <= 0;
51                  if (start) begin
52                      next_state = WRITE_FIFO;
53                  end else begin
54                      buffer_index <= 0;
55                      next_state = IDLE;
56                  end
57              end
58              WRITE_FIFO: begin
59                  s_axis_tvalid <= 0;
60                  if (write) begin
61                      buffer[buffer_index] <= din;
62
63                      if (buffer_index == (THRESHOLD-1)) begin

```

```

62         next_state = WAIT_START;
63     end else begin
64         buffer_index <= buffer_index + 1;
65         next_state = WRITE_FIFO;
66     end
67
68     end else begin
69         next_state = WRITE_FIFO;
70     end
71 end
72
73 WAIT_START: begin
74     s_axis_tvalid <= 0;
75     if (s_axis_tready) begin
76         send_index <= 0;
77         next_state = SEND;
78     end else begin
79         next_state = WAIT_START;
80     end
81 end
82
83 SEND: begin
84     if (s_axis_tready) begin
85         s_axis_tdata <= buffer[send_index];
86         s_axis_tvalid <= 1;
87
88         if (send_index == 2'd3) begin
89             next_state = DONE;
90         end else begin
91             send_index <= send_index + 1;
92             next_state = SEND;
93         end
94
95     end else begin
96         s_axis_tvalid <= 1; // Keep valid high while waiting
97         for ready
98         next_state = SEND;
99     end
100 end
101
102 DONE: begin
103     s_axis_tvalid <= 0;
104     if (s_axis_tlast) begin
105         s_axis_tdata <= {32{1'bx}};
106         next_state = IDLE;
107     end else begin
108         next_state = DONE;

```

```

108         end
109     end
110
111     default: begin
112         s_axis_tvalid <= 0;
113         next_state = IDLE;
114     end
115 endcase
116 end
117 end
118
119 endmodule

```

En 4.3 se presenta un módulo en Verilog llamado `fifo2axis`, correspondiente a la cola FIFO de conversión a AXI-Stream. Está parametrizado por el ancho de datos `DATA_WIDTH` a 32 bits, además de un umbral y la profundidad del buffer interno `DEPTH`, fijos a 4. El módulo se sincroniza con una señal de reloj `clk` y un reinicio asincrónico activo alto `rst`, el cual pasado un mínimo de tiempo, se pone a 0. Internamente, el módulo utiliza una FSM que gestiona cinco estados: `IDLE`, `WRITE_FIFO`, `WAIT_START`, `SEND` y `DONE`.

- Estado *IDLE*: En este estado, la señal `s_axis_tvalid` se mantiene inactiva (0). Si la señal de *start* se activa, la máquina transiciona al estado *WRITE_FIFO*. De lo contrario, el índice del buffer (*buffer_index*) se reinicia y permanece en *IDLE*, actuando como un estado de espera inicial.
- Estado *WRITE_FIFO*: La máquina espera a que la señal *write* se active para escribir el dato de entrada (*din*) en el buffer. Cuando se escribe, si el buffer alcanza su capacidad máxima (*THRESHOLD-1*), avanza al estado *WAIT_START*; de lo contrario, incrementa el índice y permanece aquí. La señal `s_axis_tvalid` sigue inactiva.
- Estado *WAIT_START*: Se desactiva `s_axis_tvalid` y se espera a que la señal `s_axis_tready` (receptor listo) se active. Cuando esto ocurre, se reinicia el índice de envío (*send_index*) y se pasa al estado *SEND*. Este estado sincroniza el inicio de la transmisión.
- Estado *SEND*: Se envían los datos del buffer. Cuando el receptor está listo (`s_axis_tready` en 1), se coloca el dato en `s_axis_tdata`, se activa `s_axis_tvalid` y se incrementa el índice. Si se envían todos los datos, se pasa a *DONE*; si el receptor no está listo, se mantiene `s_axis_tvalid` activo y se espera en este estado.
- Estado *DONE*: Se desactiva `s_axis_tvalid` y se espera la señal de fin de transmisión (`s_axis_tlast`). Cuando esta llega, se borran los datos (`s_axis_tdata`)

y se retorna al estado *IDLE* para una finalización limpia.

- Estado *default*: Cubre cualquier estado no definido, reiniciando la máquina a *IDLE* y desactivando `s_axis_tvalid`, garantizando un comportamiento seguro y predecible.

Listado 4.4: Código fuente en Verilog del módulo conversor de AXI al formato de datos origen

```

1 module axis2fifo #(
2     parameter DATA_WIDTH = 32,
3     parameter THRESHOLD   = 4,
4     parameter DEPTH       = 10
5 ) (
6     input  wire clk,
7     input  wire rst,
8
9     // AXI Stream master
10    input  wire [31:0] result_data,
11    input  wire      last,
12    input  wire      valid_result,
13    output reg      ready_for_results,
14
15    // To FIFO
16    output reg [31:0] dout,
17    output reg      done,
18
19    input wire      read
20 );
21
22 // States
23 localparam IDLE          = 3'd0,
24             READ_STREAM  = 3'd1,
25             SEND_OUTSIDE = 3'd2,
26             DONE         = 3'd4;
27
28 reg [2:0] state, next_state;
29 integer  i;
30
31 // Internal buffer
32 reg [31:0] buffer [0:3];
33 reg [1:0]  buffer_index;
34
35 always @(posedge clk or posedge rst) begin
36     if (rst) begin
37         state          = IDLE;
38         buffer_index    <= 2'd0;

```

```
39     done          <= 0;
40     ready_for_results <= 0;
41
42     end else begin
43         state = next_state;
44
45         case (state)
46             IDLE: begin
47                 ready_for_results <= 1;
48
49                 if (valid_result) begin
50
51                     //First value entered! We have to read it!
52                     buffer[buffer_index] <= result_data;
53                     buffer_index          <= buffer_index + 1;
54                     next_state            = READ_STREAM;
55
56                 end else begin
57                     next_state = IDLE;
58                 end
59             end
60
61             READ_STREAM: begin
62
63                 if (last == 1'b0) begin
64                     buffer[buffer_index] <= result_data;
65                     buffer_index          <= buffer_index + 1;
66                     next_state            = READ_STREAM;
67
68                 end else if (last == 1'b1) begin
69                     buffer[buffer_index] <= result_data;
70                     buffer_index          <= 0;
71                     next_state            = SEND_OUTSIDE;
72
73                 end else begin
74                     next_state = READ_STREAM;
75                 end
76             end
77
78             SEND_OUTSIDE: begin
79                 done <= 1;
80
81                 if (read) begin
82                     dout <= buffer[buffer_index];
83
84                     if (buffer_index == (THRESHOLD-1)) begin
```

```

86         next_state = DONE;
87     end else begin
88         buffer_index <= buffer_index + 1;
89     end
90
91     end else begin
92         next_state = SEND_OUTSIDE;
93     end
94 end
95
96 DONE: begin
97     dout <= {32{1'bX}};
98     done <= 0;
99     ready_for_results <= 0;
100    next_state = IDLE;
101 end
102
103 default: next_state = IDLE;
104
105 endcase
106 end
107 end
108
109 endmodule

```

En 4.4 se presenta el código fuente del módulo `axis2fifo`, que hace la función de conversor del protocolo AXI al formato original de envío de los datos. Este módulo es complementario al módulo `fifo2axis`, ya que permite recibir datos serializados de un acelerador o componente AXI y almacenarlos temporalmente en un buffer interno antes de enviarlos como los datos originales, pero con el resultado de revertir bits. El diseño está parametrizado mediante `DATA_WIDTH` a 32 bits, `THRESHOLD` (cantidad de palabras a procesar, 4) y `DEPTH`, definido a 10, aunque este último no se usa explícitamente, es únicamente el límite del buffer implementado. La lógica del módulo está controlada por una FSM con cuatro estados: `IDLE`, `READ_STREAM`, `SEND_OUTSIDE` y `DONE`.

- Estado *IDLE*: En este estado, la señal `ready_for_results` se mantiene activa (1). Si la señal `valid_result` está activa, se almacena el primer valor de `result_data` en el buffer, se incrementa el índice `buffer_index` y se transiciona al estado *READ_STREAM*. De lo contrario, la máquina permanece en *IDLE*, esperando por datos válidos.
- Estado *READ_STREAM*: Se continúa leyendo el flujo de datos. Si la señal `last` es 0 (no es el último dato), se almacena `result_data` en el buffer y se incrementa `buffer_index`, permaneciendo en este estado. Si `last` es 1 (último dato), se almacena

el dato, se reinicia *buffer_index* a 0 y se transiciona al estado *SEND_OUTSIDE*. Este estado garantiza la lectura completa del flujo de entrada.

- Estado *SEND_OUTSIDE*: Se activa la señal *done* (1) para indicar que los datos están listos para ser leídos externamente. Cuando la señal *read* está activa, se envía el dato actual del buffer a la salida *dout*. Si se alcanza el final del buffer (*THRESHOLD-1*), se transiciona al estado *DONE*; de lo contrario, se incrementa *buffer_index* para continuar enviando datos. Si *read* no está activo, se permanece en este estado.
- Estado *DONE*: Se realiza la limpieza final: la salida *dout* se establece a valores indefinidos ($\{32\{1'bX\}\}$), se desactivan las señales *done* y *ready_for_results* (0), y se transiciona al estado *IDLE*. Este estado asegura una finalización ordenada del proceso.
- Estado *default*: Cubre cualquier estado no definido, forzando una transición al estado *IDLE* para mantener un comportamiento robusto y predecible.

Listado 4.5: Fragmentos relevantes del acelerador

```

1 module bit_reverse_accel (
2     ap_clk,
3     ap_rst_n,
4     s_axis_TDATA,
5     s_axis_TVALID,
6     s_axis_TREADY,
7     s_axis_TKEEP,
8     s_axis_TSTRB,
9     s_axis_TLAST,
10    m_axis_TDATA,
11    m_axis_TVALID,
12    m_axis_TREADY,
13    m_axis_TKEEP,
14    m_axis_TSTRB,
15    m_axis_TLAST
16 );
17
18 // ...
19
20 parameter ap_ST_fsm_pp0_stage0 = 4'd1;
21 parameter ap_ST_fsm_pp0_stage1 = 4'd2;
22 parameter ap_ST_fsm_pp0_stage2 = 4'd4;
23 parameter ap_ST_fsm_pp0_stage3 = 4'd8;
24
25 (* fsm_encoding = "none" *) reg [3:0] ap_CS_fsm;
26

```

```

27 // ...
28
29 reg ap_rst_n_inv;
30 reg s_axis_TDATA_blk_n;
31 reg m_axis_TDATA_blk_n;
32 reg ap_enable_reg_pp0_iter1;
33
34 // ...
35
36 assign m_axis_TDATA = {
37     s_axis_TDATA[0], s_axis_TDATA[1], s_axis_TDATA[2],
38     s_axis_TDATA[3],
39     s_axis_TDATA[4], s_axis_TDATA[5], s_axis_TDATA[6],
40     s_axis_TDATA[7],
41     ...
42     s_axis_TDATA[31]
43 };
44 // ...

```

En 4.5 se presenta de forma resumida el código fuente generado por Vitis HLS de la función top del acelerador reverso de bits. Este módulo organiza la inversión de bits como un bloque *pipeline* con interfaz AXI-Stream, listo para integrarse en un sistema mayor. La estructura del código refleja las fases típicas de un diseño de hardware acelerado: interfaz, FSM, control de flujo y operación principal. Se ha dividido en diversas partes separadas por puntos cada uno de los apartados más importantes.

En la primera parte se definen los puertos del módulo:

- `ap_clk` y `ap_rst_n` son reloj y reset.
- Las señales `s_axis_*` representan la entrada AXI-Stream.
- Las señales `m_axis_*` representan la salida AXI-Stream.

En la segunda parte, se define la máquina de estados presente en el módulo, a juicio de la herramienta Vitis HLS; El diseño está dividido en cuatro etapas de *pipeline*, representadas como estados de la FSM. Esto permite procesar varios datos en paralelo, aprovechando el flujo continuo que ofrece AXI-Stream.

En la tercera parte, se definen las señales de control internas al módulo; estas señales sirven para manejar el control del flujo de datos:

- `ap_rst_n_inv` es el reset invertido.

- `s_axis_TDATA_blk_n` y `m_axis_TDATA_blk_n` indican bloqueo de entrada y salida.
- `ap_enable_reg_pp0_iter1` habilita la siguiente iteración en el pipeline.

En la última parte, se implementa la inversión de bits: el valor de entrada de 32 bits (`s_axis_TDATA`) se reordena de manera que el bit menos significativo pase a ser el más significativo, y viceversa. Este patrón es muy usado en algoritmos como la Fourier Fast Transform (FFT), por ejemplo.

4.3. Iteración 3 - Ejecución de simulaciones funcionales

En esta sección se detallarán las simulaciones realizadas sobre el diseño. Para realizar estas pruebas, ha sido necesario el desarrollo de un archivo de *testbench*, concepto explicado en 2.1.3. La ejecución de las simulaciones puede dividirse en dos apartados, las realizadas con Vivado y las realizadas con EdaPlayground, una aplicación web, del cual se habló en 2.1.4.

4.3.1. Testbench desarrollado

El banco de pruebas desarrollado para realizar las simulaciones, tanto en Vivado como en EdaPlayground, se presenta en 4.6

Listado 4.6: Código fuente del testbench

```
1 `timescale 1 ns / 1 ps
2
3 module tb_bitreversal;
4
5     // Señales de testbench
6     reg clk          = 1;
7     reg reset        = 1;
8     reg start_flag_i = 0;
9     reg [31:0] din_i  = {32{1'bX}};
10    reg read_i        = 0;          // read result from fifo
11    reg write_i       = 0;          // push data into fifo
12    wire [31:0] dout_o;             // output data
13    wire done_flag_o;              // checks results available
14    integer i;
```

```
15
16 // Instancia del diseño
17 bitreversal dut (
18     .clk(clk),
19     .din_i(din_i),
20     .done_flag_o(done_flag_o),
21     .dout_o(dout_o),
22     .read_i(read_i),
23     .reset(reset),
24     .start_flag_i(start_flag_i),
25     .write_i(write_i)
26 );
27
28 // Reloj
29 always #5 clk = ~clk;
30
31 initial begin
32     $dumpvars(0, tb_bitreversal); //debug waves on
33     edaplayground
34     $monitor("t=%0t din=%h dout=%h done=%b", $time, din_i,
35         dout_o, done_flag_o);
36     #10 //1 cycle
37     start_flag_i = 1;
38     reset = 0;
39     #10;
40
41 // WRITING PROCESS
42 for (i = 0; i < 4; i = i + 1) begin
43     @(posedge clk);
44     din_i = $random;
45     @(posedge clk);
46     write_i = 1;
47     #50;
48     @(posedge clk);
49     write_i = 0;
50 end
51
52 // WAIT TO FINISH PROCESSING
53 wait(done_flag_o);
54
55 // WRITING PROCESS
56 for (i = 0; i < 4; i = i + 1) begin
57     #10;
58     @(posedge clk);
59     read_i = 1;
60     #300;
61     @(posedge clk);
```

```

60         read_i = 0;
61     end
62
63     // WAIT TO FINISH RESULTS TRANSFER
64     wait(!done_flag_o);
65     #100 $finish;
66 end
67
68 endmodule

```

Este testbench valida el funcionamiento del módulo Top del diseño, llamado *bitreversal*, mediante una secuencia de operaciones sencillas. Tras inicializar las señales y generar el reloj, comienza desactivando el reset y activando la señal de inicio *start_flag_i*. A continuación, se escriben cuatro valores aleatorios en el módulo por la señal de entrada *din*, controlando la escritura mediante pulsos de la señal *write_i*. Escritos todos los datos, se espera a que el módulo termine el procesamiento interno monitoreando la señal *done_flag_o*. Cuando finalmente está a 1, los resultados están listos y procede a leerlos mediante cuatro operaciones de lectura controladas por pulsos de *read_i*, y leyendo los datos resultado a través de la señal de salida *dout_o*. Finalmente, espera a que la señal *done_flag_o* se desactive, algo que sucede cuando todos los resultados han sido leídos, espera 10 ciclos, y termina la simulación.

Simulaciones en Vivado

La herramienta Vivado, ya detallada previamente en 2.1.4, se ha empleado para realizar además simulaciones sobre el diseño, el cual fue creado dentro de esta herramienta. El diagrama de onda resultado se presenta en la figura 4.2.

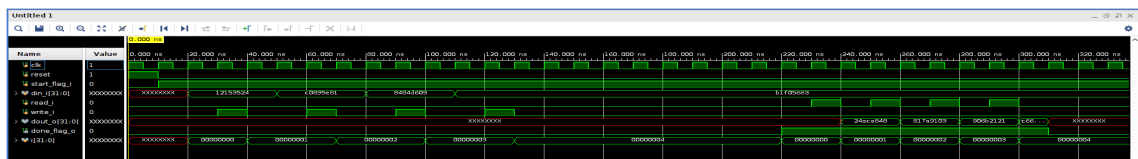


Figura 4.2: Simulación de Vivado vista con su visor integrado

El comportamiento es acorde al esperado; La simulación comienza con la señal de reloj (*clk*) oscilando de manera regular. Después, el *reset* se desactiva, permitiendo que el módulo comience a operar. Simultáneamente, la señal *start_flag_i* se activa, indicando que el proceso debe iniciar. se pulsa varias veces. Conforme avanza el tiempo, se introducen valores de datos aleatorios (por ejemplo, 12153524) en la entrada *din_i*, escribiéndose realmente estos valores en el búfer interno del módulo al subir *write_i*. Después de completar la fase de escritura, el módulo procesa los datos, lo que lleva cierto tiempo en ciclos.

Cuando La señal `done_flag_o` finalmente se activa, el procesamiento ha terminado y los resultados están listos. Una vez que `done_flag_o` está en alto, se leen datos, uno por cada pulso de la señal `read_i`. Con cada operación, los datos procesados aparecen en la salida `dout_o`, mostrando valores como `24aca848`, el resultado de aplicar la inversa de bits a la entrada `12153524`, concretamente. Cuando se han leído todos los datos, `done_flag_o` se pone a 0, indicando que la transferencia se ha completado. Después, el módulo vuelve a un estado inactivo y todas las señales se estabilizan.

Simulaciones en EdaPlayground

La herramienta web colaborativa EdaPlayground también ha sido empleada para realizar pruebas sobre el diseño desarrollado. El programa desarrollado para realizar las simulaciones en EdaPlayground es idéntico al de Vivado, presentado en 4.6, salvo por la introducción de `$dumpvars(0, tb_bitreversal)`; al inicio de la ejecución del testbench. En este trabajo, para realizar esta simulación, se ha empleado el simulador Siemens QuestaSim 2024.3, proporcionado gratuitamente por EdaPlayground.

Al igual que en Vivado, se ha realizado una simulación en EdaPlayground. EdaPlayground incluye un visor de onda propio, EPWave, pero es poco intuitivo si el tiempo de simulación es elevado. Por ello, se han descargado los resultados de la web, para poder ver el diagrama de onda en la aplicación de escritorio GTKWave, ya mencionada en 2.1.4. El resultado se presenta en la figura 4.3.

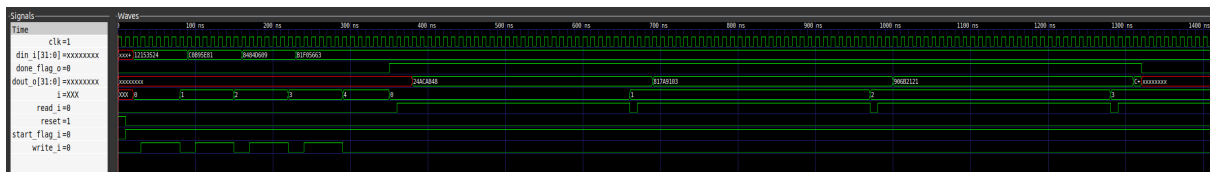


Figura 4.3: Simulación de EdaPlayground vista con GTKWave

Como puede apreciarse, el comportamiento es equivalente al obtenido con Vivado, los valores introducidos son los mismos, así como los resultados obtenidos. Esto sucede por no haber invertido tiempo en la randomización de valores para realizar las pruebas, pues se ha hecho uso de la directiva `$random` a secas, de forma que el número aleatorio será el mismo en todas las ejecuciones y determinado por el compilador.

4.4. Iteración 4 - Integración del diseño en GR-HEEP

En esta sección se detalla el proceso de integración del diseño final dentro de la plataforma GR-HEEP, como una forma más conveniente para realizar la adición de periféricos aceleradores, tal y como se expuso en 2.1.4. Como referencias, se ha tomado principalmente una asignatura del máster, Aceleradores de dominio específico, concretamente la Práctica 5, impartida por David Mallasén [Qui25a]. Para poder realizar esta integración es necesario entender dónde va a estar nuestro diseño dentro del sistema al completo. Para ello, se presenta en 4.4 un esquema aclarativo de la ubicación del acelerador dentro del conjunto global.

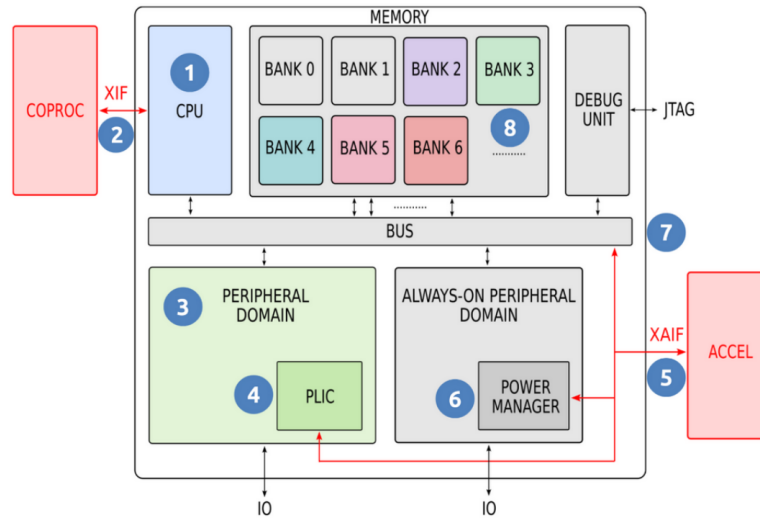


Figura 4.4: Arquitectura de GR-HEEP integrando un acelerador (5) Fuente: [Qui25a]

Como puede observarse, el acelerador está representado por el número 5, estando conectado al bus del sistema y al dominio de periféricos (tanto externos como *always-on*) mediante eXtensible Accelerator InterFace (X-AIF). X-AIF es una interfaz hardware diseñada específicamente para la plataforma X-HEEP que permite integrar aceleradores y periféricos personalizados de manera modular y estandarizada [MSM⁺24]. Al contrario que el diseño hardware que se ha desarrollado en este trabajo, X-AIF utiliza el protocolo OBI, basado en el esquema *request-response*, detallado en 2.1.3, para realizar todas las comunicaciones de la plataforma, lo que aporta ventajas importantes, como la facilidad para extender el sistema base con aceleradores multi-propósito sin tener que rediseñar la arquitectura del sistema, o un enfoque hacia la gestión de energía (*power domain*) para aplicaciones de ultra bajo consumo, si bien en este proyecto esto último queda fuera de contexto. Una muestra a mayor detalle, incluso con las señales más relevantes, se presenta en 4.5. Como aclaración, las flechas

representan el flujo de datos, y la señal correspondiente que se encarga de llevarlos. Más concretamente, una *request* es la combinación de un dato y una dirección; por ello, se pone una flecha hacia Addr, la cual está definida gracias a la generación automática de registros para el diseño, explicada en pasos posteriores, y luego otra hacia los registros, representando que se mira la dirección, y si es válida, se escribe un dato en el registro oportuno, lo que desencadenará eventos dentro del diseño, como la escritura de valores, el comienzo de envío de datos, recepción, o lectura de los resultados. De esta forma, la tarea hardware desarrollada se ejecutará.

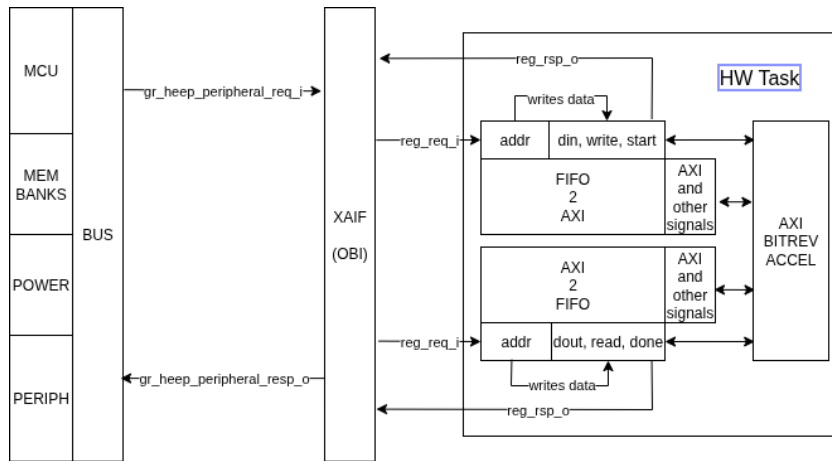


Figura 4.5: Diagrama de conexión entre X-HEEP y el acelerador a través de las FIFOs y buses

Finalmente, para poder integrar el diseño en la plataforma, es necesario seguir una serie de pasos generales:

1. Importación de los archivos del diseño
2. Conexión físico del diseño
3. Definición de los registros expuestos
4. Desarrollo de un *driver* para los registros
5. Compilación completa del sistema

Estos pasos serán detallados en posteriores secciones. Una vez completadas, el diseño estará integrado en la plataforma.

4.4.1. Importación de los archivos del diseño

En este paso se importan los archivos RTL del diseño desarrollado en la plataforma. En este contexto, hay dos formas posibles de hacerlo:

1. Manualmente: creación de un árbol de directorios como el del siguiente repositorio GitHub *simple-cnt* [Qui25c]
2. Automáticamente: uso de la herramienta *vendor* de OpenTitan, integrada en la plataforma.

Si bien se llegó a emplear la forma automática en el máster, en este proyecto se ha empleado la forma manual, por simpleza, y por ser la primera versión del diseño: en escenarios colaborativos y con control de versiones, la opción automática cobra mayor importancia. Los archivos deben estar presentes dentro de la carpeta de la plataforma, en la ruta *hw*

vendor, algo que la herramienta *vendor* hace automáticamente. El resultado de aplicar este paso se muestra en la figura 4.6.

```
(base) fluc@lights@ubuntu:~$ cd Escritorio/PERTE/TFM/GR-HEEP-connect-bus/
(base) fluc@lights@ubuntu:~/Escritorio/PERTE/TFM/GR-HEEP-connect-bus$ cd hw/vendor/bit_reversal/
(base) fluc@lights@ubuntu:~/Escritorio/PERTE/TFM/GR-HEEP-connect-bus/hw/vendor/bit_reversal$ tree -L 2
.
├── bitreversal.core
├── build
│   └── politico_isa-lab_bitreversal_1.0.0
├── fusesoc.conf
├── hw
│   ├── axis2fifo.sv
│   ├── bitreversal_obi.sv
│   ├── bitreversal.sv
│   ├── bit_reverse_accel_regslice_both.sv
│   ├── bit_reverse_accel.sv
│   ├── control-reg
│   ├── design_1.sv
│   ├── fifo2axis.sv
│   ├── misc
│   ├── packages
│   └── vendor
├── LICENSE
├── makefile
├── sw
│   ├── bitreversal_control_reg.h
│   └── bitreversal_control_reg.md
├── tb
│   ├── cnt_tb.cpp
│   ├── cnt_tb_wrapper.sv
│   ├── obi.hh
│   ├── reg.hh
│   ├── tb_components.cpp
│   ├── tb_components.hh
│   ├── tb_macros.cpp
│   ├── tb_macros.hh
│   └── waves.gtkw
├── util
│   └── vendor.py
└── 10 directories, 23 files
(base) fluc@lights@ubuntu:~/Escritorio/PERTE/TFM/GR-HEEP-connect-bus/hw/vendor/bit_reversal$
```

Figura 4.6: Importando diseño en la plataforma

Dentro de esta carpeta, se han realizado cambios en el archivo *.core* base: se ha renombrado el archivo al nombre del diseño, en este contexto, *bitreversal*. Dentro de él, se han cambiado los archivos RTL a tener en cuenta para la compilación, síntesis, y simulación del diseño en la plataforma y el campo *name* para poder hacer referencia a éste en el resto de ficheros.

4.4.2. Conexionado físico del diseño

En este paso se detalla los aspectos a tener en cuenta para conectar a nivel físico el diseño desarrollado con la plataforma, integrando el acelerador en el bus del sistema como un periférico más, utilizando la interfaz XAIF, la cual trabaja sobre el protocolo OBI.

Para este proceso, es necesario tener en cuenta dos aspectos fundamentales:

1. Al tener que adecuarse al protocolo OBI, es necesario realizar un *wrapper* (explicado en 2.1.3) que "hable" protocolo OBI. El repositorio tomado como referencia [Qui25c] viene con este fichero ya creado, y no es necesario hacer más cambios que el propio nombre del archivo y del módulo, modificados a *bitreversal_obi*.
2. Para que el sistema sea capaz de interactuar con el diseño, es necesario exponer las entradas y salidas definidas en el mismo por medio de registros con un tamaño adecuado para cada uno. Esto implica la creación de un controlador de estos registros de entrada y de salida. Este archivo también está incluido en la referencia [Qui25c], y ha sido modificado para adaptarse a los registros del diseño. Esto se explica en detalle en pasos posteriores.

Por lo tanto, el diseño a integrar, a nivel conceptual, será el presentado en 4.7.

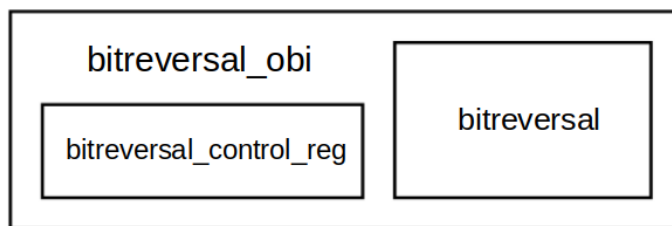


Figura 4.7: Concepto del diseño a integrar en la plataforma

Por último, para que la plataforma sea capaz de reconocer el diseño y los registros de entrada y salida, es necesario realizar a su vez, una serie de pasos, en los archivos de la carpeta raíz de GR-HEEP [Qui25a]:

1. Asignar direcciones base y tamaño en el mapa de memoria *gr-heep-cfg.hjson*, en *config/*
2. Instanciar diseño como periférico en la plantilla *gr_heap_peripherals.sv.tpl*, en *hw/peripherals/*.

4.4.3. Definición de los registros expuestos

Los registros consituyen en GR-HEEP la forma de introducir y obtener información del diseño integrado. Por tanto, para que los programas desarrollados sobre la plataforma sean capaces de interactuar con el diseño integrado dentro de la misma, es necesario representar de alguna forma los registros que se emplearán para dicho propósito.

Afortunadamente, este proceso se hace semi-automáticamente gracias a la herramienta *regtool* de OpenTitan [Ope25]. En los archivos de referencia se incluye un *script* que hace uso de esta herramienta y automatiza el proceso. Esta utilidad genera, a partir de un archivo de definición de registros *.hjson*, los esqueletos en lenguaje HDL como Verilog de las especificaciones de cada registro definido dentro de dicho archivo, además de un archivo de cabeceras *.h* que contiene las macros de las direcciones mapeadas en memoria de cada uno de los registros. Esto último es posible gracias a los cambios realizados en *gr-heap-cfg.hjson*. En el listado 4.7 se muestra el fichero de definición de registros desarrollado para este trabajo.

Listado 4.7: Archivo de definición de registros

```
1 {
2   name: "bitreversal_control",
3   clock_primary: "clk_i",
4   reset_primary: "rst_ni",
5   bus_interfaces: [
6     {
7       protocol: "reg_iface",
8       direction: "device"
9     }
10  ],
11  regwidth: "32",
12  registers: [
13    {
14      name: "DIN",
15      desc: "Operando de entrada",
16      fields: [
17        {
18          bits: "31:0",
19          name: "DIN",
20          desc: "Operando de entrada.",
21          swaccess: "rw",
22          hwaccess: "hro",
23          resval: "0"
24        }
25      ]
26    }
27  ]
28 }
```

```

26     },
27     {
28         name: "START",
29         desc: "Control de inicio de la operación",
30         fields: [
31             {
32                 bits: "0",
33                 name: "START",
34                 desc: "Escribir 1 para iniciar la operación.
35                     Permanece en 1 hasta que el software
36                     escriba 0.",
37                 swaccess: "rw",
38                 hwaccess: "hro",
39                 resval: "0"
40             }
41         ],
42     },
43     {
44         name: "READ",
45         desc: "Permite obtener dato resultado, por cada
46             activacion",
47         fields: [
48             {
49                 bits: "0",
50                 name: "READ",
51                 desc: "Escribir 1 para iniciar la operación,
52                     escribir 0 para parar",
53                 swaccess: "rw",
54                 hwaccess: "hro",
55                 resval: "0"
56             }
57         ],
58     },
59     {
60         name: "WRITE",
61         desc: "Permite encolar un dato en la FIFO, por cada
62             activacion",
63         fields: [
64             {
65                 bits: "0",
66                 name: "WRITE",
67                 desc: "Escribir 1 para iniciar la operación,
68                     escribir 0 para parar",
69                 swaccess: "rw",
70                 hwaccess: "hro",
71                 resval: "0"
72             }
73         ],
74     },
75 ]

```

```

67     ]
68     },
69     {
70         name: "DONE",
71         desc: "Estado de la operación",
72         fields: [
73             {
74                 bits: "0",
75                 name: "DONE",
76                 desc: "Se pone en 1 cuando el resultado está
                        disponible. Se borra al leerlo o
                        escribiendo 1.",
77                 swaccess: "rw1c",
78                 hwaccess: "hwo",
79                 resval: "0"
80             }
81         ]
82     },
83     {
84         name: "DOUT",
85         desc: "Resultado de la operación",
86         fields: [
87             {
88                 bits: "31:0",
89                 name: "DOUT",
90                 desc: "Resultado del cálculo (32 bits).",
91                 swaccess: "ro",
92                 hwaccess: "hwo"
93             }
94         ]
95     }
96 ]
97 }

```

Una vez generados los archivos, es necesario crear el *wrapper* que instancia los esqueletos Verilog generados. En este trabajo, se ha adaptado el archivo ya existente en el repositorio de referencia [Qui25c], siendo el resultado final presentado en el listado 4.8.

Listado 4.8: Código fuente del testbench

```

1 module bitreversal_control_reg (
2     input  logic clk_i,
3     input  logic rst_ni,
4
5     // Interfaz de registros

```

```

6  input  bitreversal_reg_pkg::reg_req_t  req_i,
7  output bitreversal_reg_pkg::reg_resp_t rsp_o,
8
9  // Señales del hardware
10 input  logic          done_i,
11 input  logic [31:0] dout_i,
12
13 output logic          start_o,
14 output logic          read_o,
15 output logic          write_o,
16 output logic [31:0] din_o
17 );
18
19 // Señales internas
20 bitreversal_control_reg_pkg::bitreversal_control_reg2hw_t
    reg2hw;
21 bitreversal_control_reg_pkg::bitreversal_control_hw2reg_t
    hw2reg;
22
23 // -----
24 // Señales escritas por HW
25 // -----
26 // DONE (rw1c/hwo): HW sets, SW clears
27 assign hw2reg.done.d  = done_i;
28 assign hw2reg.done.de = 1'b1;
29 // DOUT (ro/hwo): resultado de hardware
30 assign hw2reg.dout.d  = dout_i;
31 assign hw2reg.dout.de = 1'b1;
32
33 // -----
34 // Instancia del toplevel de registros generado
35 // -----
36 bitreversal_control_reg_top #(
37     .reg_req_t(bitreversal_reg_pkg::reg_req_t),
38     .reg_rsp_t(bitreversal_reg_pkg::reg_resp_t)
39 ) i_bitreversal_control_reg_top (
40     .clk_i      (clk_i),
41     .rst_ni     (rst_ni),
42     .reg_req_i  (req_i),
43     .reg_rsp_o  (rsp_o),
44     .reg2hw     (reg2hw),
45     .hw2reg     (hw2reg),
46     .devmode_i  (1'b0)
47 );
48
49 // -----
50 // SW Signals - Sticky Mode for simplicity

```

```

51 // (rw/hro): SW controlled, no HW override
52 // --> no assignment to hw2reg.din.* (RTL never clears it!)
53 // -----
54 assign din_o    = reg2hw.din.q;
55 assign start_o  = reg2hw.start.q;
56 assign read_o   = reg2hw.read.q;
57 assign write_o  = reg2hw.write.q;
58
59 endmodule

```

En este fichero se asignan las funcionalidades que tendrán cada uno de los registros, así como la instanciación del módulo Top del controlador. En este contexto, las salidas del diseño serán las entradas para el controlador y viceversa: la nomenclatura `__i` y `__o` es correcta en este caso.

4.4.4. Desarrollo de un *driver* para los registros

Tras la definición e integración de los registros en la plataforma, es posible desarrollar programas que hagan uso de esos registros, siendo la forma de interacción con el *hardware*. Esto se puede hacer de dos formas:

- Programar los accesos de lectura y de escritura a los registros directamente en el código de pruebas
- Desarrollar un *driver*, el cual pueda ser incluido en todos los programas que hagan uso del diseño

La opción segunda ha sido la elegida en este trabajo. Por tanto, es necesario desarrollar un *driver*, capaz de acceder a los registros para introducir u obtener valores y así interactuar con el diseño. Este *driver* deberá alojarse en la carpeta `sw/external/-lib/drivers`. En el listado 4.9 se presenta el código en cuestión.

Listado 4.9: Código fuente del controlador de bit reversal

```

1 #include <stdint.h>
2
3 #include "bitreversal.h"
4 #include "bitreversal_control_reg.h"
5 #include "csr.h"
6 #include "gr_heap.h"
7 #include "rv_plic.h"
8

```

```

9
10 void bitrev_write_val(uint32_t value) {
11     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
12     BITREVERSAL_CONTROL_DIN_REG_OFFSET) = value;
13     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
14     BITREVERSAL_CONTROL_WRITE_REG_OFFSET) = 1U; //activate
15     write
16     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
17     BITREVERSAL_CONTROL_WRITE_REG_OFFSET) = 0U; //deactivate
18     write
19 }
20
21 uint32_t bitrev_get_input() {
22     return *(volatile uint32_t *) (
23     BIT_REVERSAL_PERIPH_START_ADDRESS +
24     BITREVERSAL_CONTROL_DIN_REG_OFFSET);
25 }
26
27 void bitrev_start() {
28     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
29     BITREVERSAL_CONTROL_START_REG_OFFSET) |=
30     (1u << BITREVERSAL_CONTROL_START_START_BIT);
31 }
32
33 void bitrev_stop() {
34     *(volatile uint32_t *) (BIT_REVERSAL_START_ADDRESS +
35     BITREVERSAL_CONTROL_START_REG_OFFSET) &=
36     ~(1u << BITREVERSAL_CONTROL_START_START_BIT);
37 }
38
39 void bitrev_trigger_read() {
40     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
41     BITREVERSAL_CONTROL_READ_REG_OFFSET) = 1U;
42     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
43     BITREVERSAL_CONTROL_READ_REG_OFFSET) = 0U;
44 }
45
46 void bitrev_trigger_write() {
47     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
48     BITREVERSAL_CONTROL_WRITE_REG_OFFSET) = 1U;
49     *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
50     BITREVERSAL_CONTROL_WRITE_REG_OFFSET) = 0U;
51 }
52
53 uint8_t bitrev_is_done() {
54     return ((* (volatile uint32_t *) (
55     BIT_REVERSAL_PERIPH_START_ADDRESS +

```

```

        BITREVERSAL_CONTROL_DONE_REG_OFFSET)) >>
42         BITREVERSAL_CONTROL_DONE_DONE_BIT) & 0x1;
43     }
44
45     inline void bitrev_clear_done() {
46         *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
        BITREVERSAL_CONTROL_DONE_REG_OFFSET) =
47         (1u << BITREVERSAL_CONTROL_DONE_DONE_BIT);
48     }
49
50     uint32_t bitrev_get_output() {
51         *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
        BITREVERSAL_CONTROL_READ_REG_OFFSET) = 1U;
52         uint32_t val = *(volatile uint32_t *) (
        BIT_REVERSAL_PERIPH_START_ADDRESS +
        BITREVERSAL_CONTROL_DOUT_REG_OFFSET);
53         *(volatile uint32_t *) (BIT_REVERSAL_PERIPH_START_ADDRESS +
        BITREVERSAL_CONTROL_READ_REG_OFFSET) = 0U;
54         return val;
55     }
56
57     inline void bitrev_wait_done() {
58         while (!bitrev_is_done()) {
59             ; // busy-wait
60         }
61         bitrev_clear_done();
62     }

```

Las funciones clave, presentadas en el listado, son las siguientes:

- **uint32_t bitrev_get_output()**

Esta función obtiene el resultado de la operación de bit-reversal. Primero activa la señal **READ** escribiendo un 1 en el registro correspondiente, luego lee el valor del registro **DOUT** que contiene el resultado y finalmente desactiva la señal **READ** escribiendo un 0. Devuelve el resultado leído como un entero de 32 bits.

- **uint8_t bitrev_is_done()**

Esta función verifica si la operación de bit-reversal ha finalizado. Para ello, lee el valor del registro **DONE**, extrae el bit correspondiente y devuelve 1 si la operación está completa o 0 si aún está en proceso.

- **void bitrev_start()**

Esta función inicia la operación de bit-reversal. Escribe un 1 en el bit **START** del registro de control, lo que provoca que el módulo hardware comience a procesar el valor almacenado previamente en el registro de entrada **DIN**.

■ **void bitrev_write_val(uint32_t value)**

Esta función escribe un valor de entrada en el periférico. Primero escribe el valor dado en el registro DIN, luego activa la señal WRITE escribiendo un 1 para indicar que el dato está listo para ser procesado, y finalmente desactiva dicha señal escribiendo un 0.

4.4.5. Compilación completa del sistema

La fase de compilación en X-HEEP/GR-HEEP es un proceso automatizado que combina FuseSoC para la gestión de dependencias RTL y Verilator para la simulación del sistema completo. Esta fase es crítica para verificar la correcta integración de, en este contexto, el diseño desarrollado. FuseSoC lee los archivos .core (ej: peripherals.core, x-heap.core) para resolver la jerarquía de módulos RTL, actuando como un gestor de paquetes para código RTL, similar a lo que pip o npm son para software. Por otro lado, Verilator es el simulador que permite verificar el diseño hardware que se está integrando, realizando técnicas de *linting* y síntesis, entre otros.

Finalmente, para poder compilar la plataforma es necesario el uso de dos comandos:

1. **make gr-heap-gen**: procesa las plantillas .tpl y configuraciones .hjson para producir el sistema final con las especificaciones deseadas, configurables y variadas, pudiéndose ver las opciones con el comando *make gr-heap-gen help*.
2. **make verilator-build**: compila el diseño RTL utilizando Verilator, asegurando que no hay fallos, y genera un ejecutable que luego puede ser usado para simular el sistema con el diseño y definiciones de registros ya integradas, si todo es correcto.

En caso de éxito tras ejecutar estos dos comandos, el sistema generado tendrá finalmente integrado el diseño y los registros serán posibles de acceder programáticamente.

4.5. Iteración 5 - Ejecución del diseño en GR-HEEP sobre FreeRTOS

Tras la correcta integración del diseño en la plataforma, es posible la realización de programas en lenguajes de alto nivel (C/C++) que interactúen con el diseño por medio de los registros expuestos. El objetivo de esta iteración es el integrar el sistema operativo de tiempo real FreeRTOS, ya expuesto en 2.1.5, si bien, por simpleza, se

han hecho programas más simples para verificar funcionalidad y correcta comunicación con el diseño y registros.

4.5.1. Ejecución *baremetal*

Antes de utilizar FreeRTOS, se realizaron pruebas sin sistema operativo alguno, tan solo código C y el *driver* desarrollado para trabajar con el diseño. Para poder ejecutar este programa, es necesario compilarlo. GR-HEEP, al igual que X-HEEP, incorpora un compilador RISC-V, el cual es invocado con el comando *make app*. A este comando hay que pasarle más parámetros, quedando el comando final *make app PROJECT=<nombre_programa>TARGET=sim*, donde se define la aplicación a compilar (cuyos archivos deben estar en *sw/applications*) y el modo de ejecución, en este contexto, simulación, utilizándose el binario generado por Verilator al compilar la plataforma.

Una vez realizado el comando, en caso de éxito, es posible ejecutarlo con el comando *make verilator-run*. En este momento, el programa está ejecutándose en la plataforma. Además, a modo de depuración, supervisa la salida estándar y generando el diagrama de onda, para poder interpretar qué ocurre en todo momento. También hay que decir que existe una forma más directa de construir del diseño RTL con Verilator y ejecutar el programa C (asumiéndose previamente compilado): con el comando *make verilator-sim* es posible realizar esto.

En el listado 4.10 se muestra el código desarrollado para medir el número de ciclos empleado para ejecutar el reverso de bits utilizando el diseño integrado, y el número de ciclos para ejecutar una versión *software*, utilizando los *timers* de X-HEEP [EPF25c].

Listado 4.10: Código fuente de la versión *baremetal*

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <stdlib.h>
4  #include <time.h>
5  #include "core_v_mini_mcu.h"
6  #include "fast_intr_ctrl.h"
7  #include "bitreversal.h"
8  #include "ext_irq.h"
9  #include "csr.h"
10 #include "timer_sdk.h"
11 #include "x-heep.h"
12 #include "soc_ctrl.h"
13
14
15 // Reverse the bits of all numbers in the array - SOFTWARE //
```

```

16 static inline uint32_t reverseBits_SW() {
17     uint32_t timer_cycles;
18     // Input array of 4 random numbers
19     uint32_t nums[4] = {(rand() % 255), (rand() % 255), (rand()
20         % 255), (rand() % 255)};
21     uint32_t reversed[4]; // Array to store reversed numbers
22
23     // Get current Frequency
24     soc_ctrl_t soc_ctrl;
25     soc_ctrl.base_addr = mmio_region_from_addr((uintptr_t)
26         SOC_CTRL_START_ADDRESS);
27     uint32_t freq_hz = soc_ctrl_get_frequency(&soc_ctrl);
28
29     timer_cycles_init(); // Init the timer SDK for clock
30     cycles
31     timer_start(); // Start counting the time
32
33     // SOFTWARE BIT REVERSAL
34     for (size_t i = 0; i < 4; i++) {
35         uint32_t num = nums[i];
36         uint32_t reversed_num = 0;
37
38         for (int j = 0; j < 32; j++) {
39             reversed_num <= 1; // Shift left
40             reversed_num |= (num & 1); // Get the last bit of
41             num and set it in reversed_num
42             num >>= 1; // Shift num right
43         }
44         reversed[i] = reversed_num; // Store the reversed
45         number
46     }
47
48     timer_cycles = timer_stop();
49     printf("SW Input: {0x%08X, 0x%08X, 0x%08X, 0x%08X}\n",
50         nums[0], nums[1], nums[2], nums[3]);
51     printf("SW Reversed: {0x%08X, 0x%08X, 0x%08X, 0x%08X}\n",
52         reversed[0], reversed[1], reversed[2], reversed[3]);
53     return timer_cycles;
54 }
55
56 // Reverse the bits of all numbers in the array - HARDWARE //
57 static inline uint32_t reverseBits_HW() {
58
59     uint32_t timer_cycles;
60     // Input array of 4 random numbers
61     uint32_t nums[4] = {(rand() % 255), (rand() % 255), (rand()
62         % 255), (rand() % 255)};

```

```

57     uint32_t reversed[4]; // Array to store reversed numbers
58
59     // Get current Frequency
60     soc_ctrl_t soc_ctrl;
61     soc_ctrl.base_addr = mmio_region_from_addr((uintptr_t)
        SOC_CTRL_START_ADDRESS);
62     uint32_t freq_hz = soc_ctrl_get_frequency(&soc_ctrl);
63
64     timer_cycles_init(); // Init the timer SDK for clock
        cycles
65     timer_start(); // Start counting the time
66
67     // HARDWARE BIT REVERSAL
68     bitrev_start();
69
70     for (size_t i = 0; i < 4; i++) {
71         bitrev_write_val(nums[i]);
72     }
73
74     // Wait until done flag is 1 (polling)
75     while (!bitrev_is_done());
76
77     for (size_t i = 0; i < 4; i++) {
78         reversed[i] = bitrev_get_output(); // Get result
79     }
80
81     timer_cycles = timer_stop();
82     printf("HW Input: {0x%08X, 0x%08X, 0x%08X, 0x%08X}\n",
        nums[0], nums[1], nums[2], nums[3]);
83
84     printf("HW Reversed: {0x%08X, 0x%08X, 0x%08X, 0x%08X}\n",
        reversed[0], reversed[1], reversed[2], reversed[3]);
85
86     return timer_cycles;
87 }
88
89 int main() {
90     uint32_t sw_cycles, hw_cycles;
91     sw_cycles = reverseBits_SW();
92     hw_cycles = reverseBits_HW();
93     printf("SW Cycles: %d\n", sw_cycles);
94     printf("HW Cycles: %d\n", hw_cycles);
95 }

```

El objetivo del programa no es otro que hacer las versiones *hardware* y *software* del reverso de bits, e imprimir el número de ciclos requerido por cada una; en 4.8 se presenta el resultado.

```

SW Input: {0x000000CB, 0x0000004E, 0x0000003E, 0x000000A4}
SW Reversed: {0xD3000000, 0x72000000, 0x7C000000, 0x25000000}
HW Input: {0x00000000, 0x00000030, 0x0000009D, 0x000000AB}
HW Reversed: {0x00000000, 0x0C000000, 0xB9000000, 0xD5000000}
SW Cycles: 1202
HW Cycles: 235

```

Figura 4.8: Resultado de la ejecución *baremetal*

4.5.2. Ejecución sobre FreeRTOS

Para utilizar el sistema operativo de tiempo real FreeRTOS, detallado en 2.1.5, se ha seguido la documentación disponible [EPF25b], que, en resumen, es, realizar la compilación del ejemplo *example_blinky_freertos*, que contiene las cabeceras necesarias y la base para poder hacer uso del sistema operativo. De esta forma, se generan los ficheros de cabeceras utilizados por FreeRTOS en la compilación, y así poder hacer uso del mismo dentro del programa desarrollado. A la hora de desarrollar el programa que hará la ejecución del reverso de bits con el diseño *hardware* y la versión *software*, tendrán que agregarse estas cabeceras también.

Como nota adicional, se ha tenido que aumentar el número de bancos de memoria del sistema a 4; si no se realiza este paso, al realizar la compilación del programa se obtienen errores relativos al desbordamiento del *heap*. Esto puede hacerse tanto al ejecutar *make gr-heep-gen* agregando *MEMORY_BANKS=4* o cambiando a 4 el valor del campo *code_and_data* del fichero *config/mcu-gen-system.hjson* [Qui25a]. El código desarrollado se presenta en el listado 4.11

Listado 4.11: Código fuente de la versión FreeRTOS

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <time.h>
5 #include <unistd.h>
6 #include <assert.h>
7
8 #include "csr.h"
9 #include "hart.h"
10 #include "handler.h"
11 #include "core_v_mini_mcu.h"
12 #include "rv_timer.h"
13 #include "soc_ctrl.h"
14 #include "gpio.h"
15 #include "x-heep.h"
16 #include "fast_intr_ctrl.h"

```

```
17 #include "ext_irq.h"
18 #include "timer_sdk.h"
19
20 /* FreeRTOS kernel includes */
21 #include <FreeRTOS.h>
22 #include <task.h>
23 #include <queue.h>
24 #include "semphr.h"
25
26 /* HW Design includes */
27 #include "bitreversal.h"
28
29 // Hooks required by FreeRTOS config (empty implementations)
30 void vApplicationTickHook(void) {}
31 void vApplicationIdleHook(void) {}
32 void vApplicationMallocFailedHook(void) { for(;;); }
33 void vApplicationStackOverflowHook(TaskHandle_t xTask, char *
    pcTaskName) {
34     (void)xTask; (void)pcTaskName; for(;;);
35 }
36
37 /* Allocate heap to special section. Note that we have no
    references in the whole program to this variable (since its
    just here to allocatEspace in the section for our heap) */
38 __attribute__((section(".heap"), used)) uint8_t ucHeap[
    configTOTAL_HEAP_SIZE];
39
40 // Data structure for messages (can be the same for both
    directions or different)
41 typedef struct {
42     int type;
43     uint32_t cycles;           // number of cycles
44     uint32_t data[4];         // 32bit set of 4 numbers
45 } Packet_t;
46
47 // Queue handler
48 static QueueHandle_t xQueue = NULL;
49
50 Packet_t reverseBits_SW(Packet_t received);
51 Packet_t reverseBits_HW(Packet_t received);
52
53 // Reverse the bits of all numbers in the array - SOFTWARE //
54 Packet_t reverseBits_SW(Packet_t received) {
55
56     Packet_t result;
57     result.type = received.type;
```

```

59 // Get current Frequency
60 soc_ctrl_t soc_ctrl;
61 soc_ctrl.base_addr = mmio_region_from_addr((uintptr_t)
    SOC_CTRL_START_ADDRESS);
62 uint32_t freq_hz = soc_ctrl_get_frequency(&soc_ctrl);
63
64 timer_cycles_init(); // Init the timer SDK for clock
    cycles
65 timer_start(); // Start counting the time
66
67 // SOFTWARE BIT REVERSAL
68 for (int i = 0; i < 4; i++) {
69     uint32_t num = received.data[i];
70     uint32_t reversed_num = 0;
71
72     for (int j = 0; j < 32; j++) {
73         reversed_num <=< 1; // Shift left
74         reversed_num |= (num & 1); // Get the last bit
            of num and set it in reversed_num
75         num >>= 1; // Shift num right
76     }
77     result.data[i] = reversed_num; // Store the
        reversed number
78 }
79
80 result.cycles = timer_stop();
81
82 return result;
83 }
84
85 // Reverse the bits of all numbers in the array - HARDWARE //
86 Packet_t reverseBits_HW(Packet_t received) {
87
88     Packet_t result;
89     result.type = received.type;
90
91     // Get current Frequency
92     soc_ctrl_t soc_ctrl;
93     soc_ctrl.base_addr = mmio_region_from_addr((uintptr_t)
        SOC_CTRL_START_ADDRESS);
94     uint32_t freq_hz = soc_ctrl_get_frequency(&soc_ctrl);
95
96     timer_cycles_init(); // Init the timer SDK for clock
        cycles
97     timer_start(); // Start counting the time
98
99     // HARDWARE BIT REVERSAL

```

```
100     bitrev_start();
101
102     for (int i = 0; i < 4; i++) {
103         bitrev_write_val(received.data[i]);
104     }
105
106     // Wait until done flag is 1 (polling)
107     while (!bitrev_is_done());
108
109     for (int i = 0; i < 4; i++) {
110         result.data[i] = bitrev_get_output();    // Get result
111     }
112
113     result.cycles = timer_stop();
114     return result;
115 }
116
117 void vSenderTask(void *pvParameters)
118 {
119     Packet_t input;
120
121     // Send a message to the receiver
122     for (int i = 0; i < 3; i++)
123     {
124         // Initial values
125         input.type = rand() % 2;
126         input.cycles = 0; //init cycle number
127         input.data[0] = (rand() % 255);
128         input.data[1] = (rand() % 255);
129         input.data[2] = (rand() % 255);
130         input.data[3] = (rand() % 255);
131         printf("Sent: {0x%08X, 0x%08X, 0x%08X, 0x%08X}\n", input
            .data[0], input.data[1], input.data[2], input.data[3]);
132         if (xQueueSend(xQueue, &input, portMAX_DELAY) == pdPASS)
133             ;
134     }
135
136 void vReceiverTask(void *pvParameters)
137 {
138     Packet_t received;
139     Packet_t result;
140     char o; //store operation type
141
142     for (int i = 0; i < 3; i++)
143     {
144         // Receive a message from sender
145         if (xQueueReceive(xQueue, &received, portMAX_DELAY) ==
```

```

145         pdPASS)
146     {
147         if (received.type == 0) {
148             o = 'H';
149             result = reverseBits_HW(received);
150         }else {
151             o = 'S';
152             result = reverseBits_SW(received);
153         }
154         printf("Processed: type %c, input %d: {0x%08X, 0x%08X, 0x%08X, 0x%08X}, cycles:%d\n", o, i, result.data[0], result.data[1], result.data[2], result.data[3], result.cycles);
155     }
156 }
157
158 int main() {
159
160     xQueue = xQueueCreate(3, sizeof(Packet_t));
161
162     if (xQueue != NULL)
163     {
164         xTaskCreate(vSenderTask, "Sender", 300, NULL, 3, NULL);
165         xTaskCreate(vReceiverTask, "Receiver", 300, NULL, 3, NULL);
166         vTaskStartScheduler();
167     }
168     for (;;)
169 }

```

En el código, se crean dos tareas: *vSenderTask*, que envía los datos de entrada por la cola *xQueue*, pidiendo que el reverso de bits sea por hardware (representado por 0), y software (representado por 1). Estos datos son recibidos por una segunda tarea, *vReceiverTask*, que está a la escucha de *xQueue* y que realiza el reverso de bits de una u otra forma. Una vez enviados 3 conjuntos de valores por parte de *vSenderTask*, *vReceiverTask* procede a generar el resultado para cada uno, obtiene el número de ciclos empleados para realizar la operación, y muestra el resultado por salida estándar. En 4.9 se muestra el resultado fruto de la ejecución de este programa.

```

Sent: {0x0000004E, 0x0000003E, 0x000000A4, 0x00000000}
Sent: {0x0000009D, 0x000000AB, 0x00000065, 0x0000006D}
Sent: {0x000000BF, 0x000000E4, 0x000000B9, 0x0000003F}
Processed: type S, input 0: {0x72000000, 0x7C000000, 0x25000000, 0x00000000}, cycles:1075
Processed: type H, input 1: {0xB9000000, 0xD5000000, 0xA6000000, 0xB6000000}, cycles:213
Processed: type S, input 2: {0xFD000000, 0x27000000, 0x9D000000, 0xFC000000}, cycles:1074

```

Figura 4.9: Resultado de la ejecución sobre FreeRTOS

4.6. Iteración 6 - Comparativa de resultados frente a versión software

Como ya se mencionó en la iteración anterior, los programas desarrollados 4.10 y 4.11 realizan un conteo de ciclos en las versiones hardware y software. En todo momento se aprecia que la relación de aceleración entre software y hardware se mantiene constante en ambos escenarios, en torno a un factor de cinco veces. Esto confirma que el diseño del acelerador hardware es correcto y eficiente, si bien, este diseño se encuentra en una primera versión y hay un considerable margen de mejora.

Sin embargo, tal y como se observa en los resultados mostrados en las figuras 4.8 y 4.9, se puede apreciar unos valores que difieren, en cierta medida de lo lógico: la versión *baremetal* debería ser más rápida en todo momento que la versión FreeRTOS. Algo que, en este contexto, no sucede. Esta problemática puede deberse a ciertos factores, entre otros:

- En primer lugar, aunque en ambos programas se utiliza el mismo mecanismo de conteo de ciclos proporcionado por el timer-sdk de X-HEEP, el estado previo del periférico no es el mismo: en *baremetal*, el temporizador se inicializa y arranca *desde cero* en cada función de medición, lo cual introduce una latencia adicional asociada a la primera configuración y habilitación del hardware. Sin embargo, en FreeRTOS, el kernel ya mantiene el temporizador activo para el control de ticks y planificación de tareas, por lo que las llamadas a *timer_cycles_init()* y *timer_start()* recaen sobre un periférico ya en funcionamiento, reduciendo el coste de inicialización.
- En segundo lugar, debe considerarse el efecto de las librerías y la memoria. En *baremetal*, las funciones como *rand()* y *printf()* se utilizan por primera vez en las rutinas de prueba, lo que implica un mayor número de accesos a memoria y posibles penalizaciones de cache en su primera ejecución. En FreeRTOS, en cambio, antes de llegar a la tarea de medición, ya se han ejecutado varias operaciones que utilizan estas mismas librerías y estructuras, de manera que ya las inicializó, lo que reduce el coste en ciclos durante la medición.
- Por último, el modelo de simulación con Verilator puede influir. En *baremetal*, las mediciones se realizan inmediatamente después del arranque del sistema, en un estado inicial en el que las configuraciones de periféricos todavía incurren en costes adicionales. En FreeRTOS, por el contrario, el simulador Verilator ya ha ejecutado múltiples ciclos asociados al arranque del kernel, la creación de tareas y la comunicación mediante colas, de forma que al momento de realizar la medición el sistema está más estabilizado y los tiempos reportados son menores. Esto se puede observar en que el tiempo de ejecutar simulaciones sobre FreeRTOS aumenta de forma desproporcional frente a *baremetal*.

Capítulo 5

Conclusiones y trabajo futuro

En este capítulo se abordan las competencias adquiridas y se evalúan el cumplimiento de los requisitos o problemas que han impedido alcanzarlos, además de una valoración crítica del proyecto.

5.1. Competencias adquiridas

- *Capacidad de diseñar, depurar y programar sistemas integrados multiprocesador complejos.*

Esta competencia se ha desarrollado gracias a la integración de un acelerador hardware dentro de la plataforma X-HEEP/GR-HEEP, basada en la arquitectura RISC-V. El trabajo requirió comprender y aplicar metodologías de diseño para SoCs heterogéneos, abordando la interconexión entre múltiples procesadores, periféricos y módulos hardware en un entorno de gran complejidad.

- *Capacidad de generar el middleware y software de sistemas integrados adaptados a su arquitectura.*

El proyecto incluye la creación de drivers y la adaptación del diseño hardware para su correcta gestión a través de FreeRTOS. Esta labor permitió asegurar la comunicación entre el hardware acelerador y el sistema operativo en tiempo real, constituyendo un ejemplo claro de integración hardware-software en sistemas embebidos.

- *Capacidad de verificar y testear circuitos integrados utilizando diferentes tecnologías y herramientas.*

Se ha adquirido experiencia en la verificación del diseño mediante el uso de diversas herramientas (Vivado, Vitis HLS, EDAPlayground, GTKWave). Asimismo, se realizaron simulaciones funcionales, pruebas de integración, y ejecuciones comparativas de rendimiento frente a versiones software, lo que permite garantizar la validez y eficiencia del sistema implementado.

5.2. Revisión de los objetivos

A continuación, se detallan los objetivos planteados en 3.2 y cómo han sido cumplidos en el desarrollo del mismo:

- Para el objetivo principal, la *implementación e integración en GR-HEEP de una tarea hardware sobre el sistema operativo de tiempo real FreeRTOS*: se considera **cumplido**, gracias al desarrollo de un acelerador del reverso de bits, integrado a su vez en el flujo de diseño de Vivado para generar un diseño hardware que pudiera realizar la tarea en cuestión, formado por el acelerador y dos colas FIFO, de entrada y salida de datos respectivamente, e integrado como periférico externo a la plataforma X-HEEP/GR-HEEP, sobre la cual se han agregado las librerías necesarias para utilizar el sistema operativo de tiempo real FreeRTOS y ejecutar programas que realicen la tarea hardware sobre éste.
- Para el objetivo parcial: la *utilización del protocolo AXI-Stream para la entrada y salida de datos del acelerador*, se ha cumplido gracias a que al desarrollar el acelerador por medio de Vitis HLS y sobre C/C++, es posible importar la librería *hls::stream* y hacer uso de este protocolo posteriormente tras generar el código RTL gracias a la síntesis de alto nivel, e integrarlo fácilmente en Vivado con el diseño global que permita hacer la tarea en cuestión.
- Para el objetivo parcial: el *desarrollo de drivers que permitan la comunicación hardware-software*, también se considera **cumplido**, ya que se ha implementado y validado un driver específico que es capaz de leer y escribir en los registros del diseño integrado, facilitando la interacción entre el diseño contenedor del acelerador en cuestión y del software desarrollado para la ejecución en *baremetal* como en FreeRTOS.
- Para el objetivo parcial: la *integración de la infraestructura hardware desarrollada en GR-HEEP*, se da por cumplido gracias a la comprobación de la correcta ejecución de los programas desarrollados en C que realizan la tarea hardware en cuestión, así como la comprobación de los valores generados en las formas de onda fruto de la simulación con Verilator.
- Para el objetivo parcial: la *validación la infraestructura hardware mediante simulaciones y ejecuciones*, se considera parcialmente cumplido, debido a que, si bien se han desarrollado simulaciones y ejecuciones utilizando diversas herramientas como Vivado, Vitis HLS, EDAPlayground, GTKWave o Verilator, no se ha llegado a realizar una verificación extensiva del diseño, empleando metodologías como Universal Verification Methodology (UVM), debido a la falta de tiempo.
- Para el objetivo parcial: la *comparativa de rendimiento entre la tarea hardware y una tarea software del mismo tipo*, se da por **cumplido** por haberse realizado

una evaluación experimental comparando el tiempo de ejecución del algoritmo en software con la versión acelerada en hardware, además de dar justificación a los resultados obtenidos.

5.3. Valoración crítica del proyecto

5.3.1. Líneas de trabajo futuro

Como extensiones futuras a este proyecto se destaca, entre otros:

- *El uso de metodologías de verificación*, como UVM, que permitan validar el diseño y realizar un estudio de todos los escenarios posibles a nivel de código, lo que se conoce como *code coverage*
- *La reutilización de IPs ya existentes en Vivado*: en este trabajo no se llegó a realizar debido a que se consideró más interesante desarrollar todo desde cero, ya que eso involucra un mayor uso de lenguajes HDL como Verilog, estudiados en este máster.
- *Potenciales reducciones de la lógica de control*: debido a que se buscaba lograr la funcionalidad, y no tanto, la eficiencia, la lógica de control tiene un considerable margen de reducción, algo que, por falta de tiempo, no se ha podido realizar.

5.3.2. Valoración personal

El haber realizado este TFM me ha aportado más de lo que creía en un principio, destacando el potencial de la síntesis de alto nivel de aceleradores procedentes de la herramienta Vitis HLS, el uso de los Block Design en Vivado para generar un diseño hardware completo con módulos RTL de todo tipo. También me ha resultado muy interesante la capacidad de X-HEEP/GR-HEEP para la integración del diseño, donde, aún teniendo ciertas nociones gracias a varias sesiones prácticas del máster, he aprendido bastante más de lo esperado. Quiero mencionar a David Mallasén Quintana, quien me ha ayudado mucho vía correo electrónico en esta problemática. En definitiva, este proyecto me ha permitido crecer en conocimientos que, en otro contexto, me habría sido más difícil, si no imposible, de aprender.

Apéndice A

Anexo A

En este Anexo se presenta código desarrollado de menor relevancia que el presentado en las diferentes secciones del documento.

A.1. Código RTL de la primera versión del diseño

Listado A.1: Código fuente de la cola FIFO de entrada de datos

```
1 module fifo_in (
2     input wire      clk,
3     input wire      rst,          // Synchronous reset
4     input wire      wr_en,
5     input wire      rd_en,       // External read request
6     input wire [31:0] din,
7     output reg [31:0] dout,
8     output wire      full,
9     output wire      empty,
10    output reg       data_ready  // Indicates 4 words
11    available
12 );
13
14 // Parameters
15 localparam DEPTH = 4;
16 localparam ADDR_WIDTH = $clog2(DEPTH);
17 localparam THRESHOLD = 4; // 4 words!
18
19 // FSM state encoding
20 localparam IDLE = 2'b00;
21 localparam READ = 2'b01;
```



```

66
67         dout <= mem[rd_ptr];
68         rd_ptr <= rd_ptr + 1;
69
70         if (rd_ptr == THRESHOLD) begin
71             state <= WAIT;
72         end
73
74         end else begin
75             dout <= {32{1'bX}};
76             data_ready <= 0;
77         end
78     end
79
80     WAIT: begin
81         fifo_data_count <= 0;
82         wr_ptr <= 0;
83         rd_ptr <= 0;
84
85         // Set all bits to X into the internal
86         // memory FIFO
87         for (i = 0; i < THRESHOLD; i = i+1) begin
88             mem[i] <= {32{1'bX}};
89         end
90
91         state <= IDLE;
92     end
93
94     default: begin
95         state <= IDLE;
96     end
97 endcase
98 end
99 endmodule

```

Listado A.2: Código fuente de la cola FIFO conversora a AXI-Stream

```

1 module fifo2axis #(
2     parameter DATA_WIDTH = 32,
3     parameter THRESHOLD = 4,
4     parameter DEPTH = 10
5 )(
6     input  wire      clk,
7     input  wire      rst,
8

```

```

9      // FIFO interface
10     input wire [31:0] fifo_data,
11     input wire          data_ready,
12
13     // AXI Stream master
14     output reg [31:0] s_axis_tdata,
15     output reg          s_axis_tvalid,
16     input wire          s_axis_tready,
17     output wire          start_hls,
18     input wire          s_axis_tlast
19 );
20
21     // States
22     localparam IDLE      = 3'd0,
23                 READ_FIFO = 3'd1,
24                 WAIT_START = 3'd2,
25                 SEND      = 3'd3,
26                 DONE      = 3'd4;
27
28     reg [2:0] state, next_state;
29
30     // Internal buffer
31     reg [31:0] buffer [0:3];
32     reg [1:0]  buffer_index;
33     reg [1:0]  send_index;
34     reg start_hls_module;
35
36     // When RESET signal is 0 and module starts, also start the
37     // HLS module
38     assign start_hls = ~rst;
39
40     // FSM Sequential
41     always @(posedge clk or posedge rst) begin
42         if (rst) begin
43             state <= IDLE;
44             buffer_index <= 2'd0;
45             send_index <= 2'd0;
46             s_axis_tvalid <= 0;
47             s_axis_tdata <= 32'd0;
48
49         end else begin
50             state = next_state;
51
52             case (state)
53                 IDLE: begin
54                     if (data_ready) begin
55                         next_state <= READ_FIFO;
56                     end
57                 end
58             endcase
59         end
60     end

```

```

55         end else begin
56             next_state <= IDLE;
57             buffer_index <= 0;
58         end
59     end
60
61     READ_FIFO: begin
62         buffer[buffer_index] <= fifo_data;
63
64         if (buffer_index == 2'd3) begin
65             next_state <= WAIT_START;
66         end else begin
67             buffer_index <= buffer_index + 1;
68             next_state <= READ_FIFO;
69         end
70     end
71
72     WAIT_START: begin
73         if (s_axis_tready) begin
74             send_index <= 0;
75             next_state <= SEND;
76         end else begin
77             next_state <= WAIT_START;
78         end
79     end
80
81     SEND: begin
82
83         #5; //wait a bit for AXISTREAM MODULE
84
85         if (s_axis_tready) begin
86             s_axis_tdata <= buffer[send_index];
87             s_axis_tvalid <= 1;
88             #10
89             s_axis_tvalid <= 0;
90
91             if (send_index == 2'd3) begin
92                 next_state = DONE;
93             end else begin
94                 send_index <= send_index + 1;
95                 next_state <= SEND;
96             end
97
98         end else begin
99             next_state <= SEND;
100         end
101

```

```

102         end
103
104
105         DONE: begin
106             if (s_axis_tlast) begin
107                 s_axis_tdata = {32{1'bx}};
108                 next_state <= IDLE;
109             end else begin
110                 next_state <= DONE;
111             end
112         end
113
114         default: begin
115             next_state <= IDLE;
116         end
117     endcase
118 end
119 end
120
121 endmodule

```

Listado A.3: Código fuente de la cola FIFO convertora a formato original

```

1 module axis2fifo #
2 (
3     parameter DATA_WIDTH = 32,
4     parameter THRESHOLD = 4,
5     parameter DEPTH = 10
6 )(
7     input  wire      clk,
8     input  wire      rst,
9
10    // AXI Stream master
11    input  wire [31:0] s_axis_tdata,
12    input  wire      last,
13
14    // To FIFO
15    output reg [31:0] dout,
16    output reg      done
17 );
18
19    // States
20    localparam IDLE      = 3'd0,
21                READ_STREAM = 3'd1,
22                SEND_OUTSIDE = 3'd2,
23                DONE      = 3'd4;

```

```

24
25     reg [2:0] state, next_state;
26 integer i;
27 // Internal buffer
28 reg [31:0] buffer [0:3];
29 reg [1:0]  buffer_index;
30
31 always @(posedge clk or posedge rst) begin
32     if (rst) begin
33         state <= IDLE;
34         buffer_index <= 2'd0;
35         i <= 0;
36         done <= 0;
37
38     end else begin
39         state = next_state;
40
41         case (state)
42             IDLE: begin
43
44                 if (s_axis_tdata !== {32{1'bx}} &&
45                     s_axis_tdata !== {32{1'b0}}) begin
46                     buffer[buffer_index] <= s_axis_tdata;
47                     buffer_index <= buffer_index + 1;
48                     next_state <= READ_STREAM;
49                 end else begin
50                     next_state <= IDLE;
51                 end
52             end
53
54             READ_STREAM: begin
55                 if(last == 1'b0) begin
56                     buffer[buffer_index] <= s_axis_tdata;
57                     buffer_index <= buffer_index + 1;
58                     next_state <= READ_STREAM;
59
60                 end else if (last == 1'b1) begin
61                     buffer[buffer_index] <= s_axis_tdata;
62                     buffer_index <= 0;
63                     next_state <= SEND_OUTSIDE;
64                     done <= 1;
65
66                 end else begin
67                     next_state <= READ_STREAM;
68                 end
69             end
70         end

```

```

70
71         SEND_OUTSIDE: begin
72             dout <= buffer[buffer_index];
73
74             if (buffer_index == THRESHOLD -1) begin
75                 next_state <= DONE;
76                 done <= 0;
77             end else begin
78                 buffer_index <= buffer_index + 1;
79             end
80         end
81
82         DONE: begin
83             // Set output to X, all data transfered
84             dout <= {32{1'bX}};
85
86             // Set all bits to X into the internal
87             // memory FIFO
88             for (i = 0; i < THRESHOLD; i = i+1) begin
89                 buffer[i] <= {32{1'bX}};
90             end
91
92             default: begin
93                 next_state <= IDLE;
94             end
95         endcase
96     end
97 end
98
99 endmodule

```

Listado A.4: Código fuente de la cola FIFO de salida de datos

```

1 module fifo_out (
2     input wire      clk,
3     input wire      rst,          // Synchronous reset
4     input wire [31:0] din,
5     input wire      data_ready,   // Indicates 4 words available
6     output wire      full,
7     output wire      empty,
8     output reg [31:0] dout
9
10 );
11
12 // Parameters

```

```

13     localparam DEPTH = 4;
14     localparam ADDR_WIDTH = $clog2(DEPTH);
15     localparam THRESHOLD = 4; // 4 words!
16
17     // FSM state encoding
18     localparam IDLE = 2'b00;
19     localparam READ = 2'b01;
20     localparam WAIT = 2'b10;
21     reg [1:0] state;
22
23     // Memory
24     reg [31:0] mem [0:DEPTH-1];
25     reg [ADDR_WIDTH-1:0] rd_ptr = 0;
26     reg [ADDR_WIDTH-1:0] wr_ptr = 0;
27
28     // Flags
29     reg can_read;
30     integer i;
31     reg [3:0] fifo_data_count;
32
33     assign full          = (fifo_data_count == DEPTH);
34     assign empty         = (fifo_data_count == 0);
35
36     always @(posedge clk or posedge rst) begin
37
38         if (rst) begin
39             state <= IDLE;
40             wr_ptr <= 0;
41             rd_ptr <= 0;
42             fifo_data_count <= 0;
43             can_read <= 0;
44
45         end else begin
46             case (state)
47                 IDLE: begin
48
49                     if(data_ready) begin
50                         mem[wr_ptr] <= din;
51                         wr_ptr <= wr_ptr + 1;
52                         fifo_data_count <= fifo_data_count + 1;
53
54                         if (fifo_data_count >= THRESHOLD) begin
55                             fifo_data_count <= 0;
56                             can_read <= 1;
57                             state <= READ;
58                         end
59

```

```
60         end
61     end
62
63     READ: begin
64         if (can_read) begin
65
66             dout <= mem[rd_ptr];
67             rd_ptr <= rd_ptr + 1;
68
69             if (rd_ptr == THRESHOLD) begin
70                 state <= WAIT;
71             end
72
73         end else begin
74             dout <= {32{1'bX}};
75         end
76     end
77
78     WAIT: begin
79         fifo_data_count <= 0;
80         wr_ptr <= 0;
81         rd_ptr <= 0;
82
83         // Set all bits to X into the internal
84         // memory FIFO
85         for (i = 0; i < THRESHOLD; i = i+1) begin
86             mem[i] <= {32{1'bX}};
87         end
88
89         state <= IDLE;
90     end
91
92     default: begin
93         state <= IDLE;
94     end
95 endcase
96 end
97 endmodule
```


Referencias bibliográficas

- [AF15] Suliman Al Freidi. A unified project management methodology (upmm) based on pmbok and prince2 protocols: foundations, principles, structures and benefits of the integrated approach. *International Journal of Business Policy and Strategy Management*, 2:27–38, 11 2015.
- [AMD22] AMD. Digilent nexys a7. Recurso web: <https://www.amd.com/es/corporate/university-program/aup-boards/digilent-nexys-a7.html>, 2022. Accedido: 11-08-2025.
- [AMD25a] AMD. Amd vitis hls empowers rtl designers with faster verification and design iteration. Recurso web: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html>, 2025. Página principal Vitis HLS. Accedido: 14-08-2025.
- [AMD25b] AMD. Amd vitis hls user guide (ug1399). Recurso web: <https://docs.amd.com/r/en-US/ug1399-vitis-hls>, 2025. Accedido: 14-08-2025.
- [AMD25c] AMD. Amd vivado design suite. Recurso web: <https://www.amd.com/es/products/software/adaptive-socs-and-fpgas/vivado.html>, 2025. Página de descarga. Accedido: 10-08-2025.
- [AMD25d] AMD. Versal adaptive soc design guide (ug1273). Recurso web: <https://docs.amd.com/r/en-US/ug1273-versal-acap-design/System-Architecture>, 2025. Accedido: 11-08-2025.
- [AMD25e] AMD. Vitis high-level synthesis user guide (ug1399): How axi4-stream works. Recurso web: <https://docs.amd.com/r/en-US/ug1399-vitis-hls/How-AXI4-Stream-Works>, 2025. Accedido: 25-08-2025.
- [ARM23] ARM. Amba axi and ace protocol specification. Recurso web: <https://documentation-service.arm.com/static/64819f1516f0f201aa6b963c>, 2023. Accedido: 25-08-2025.

- [Auf19] Jean-Luc Aufranc. Freertos kernel now supports risc-v architecture. Recurso web: <https://www.cnx-software.com/2019/03/06/amazon-freertos-risc-v/>, 2019. Accedido: 10-08-2025.
- [Awa22] Rahul Awati. What is an intellectual property core (ip core)? Recurso web: <https://www.techtarget.com/whatis/definition/IP-core-intellectual-property-core>, 2022. TechTarget. Accedido: 14-08-2025.
- [Chi22] ChipVerify. What are top-level modules? Recurso web: <https://www.chipverify.com/verilog/verilog-modules>, 2022. Accedido: 15-08-2025.
- [Com25] Wikipedia Community. Logo de freertos. Recurso web: https://es.wikipedia.org/wiki/FreeRTOS#/media/Archivo:Logo_freeRTOS.png, 2025. Accedido: 10-08-2025.
- [Cor21] Intel Corporation. An 917: Reset design techniques for hyperflex architecture fpgas. Recurso web: <https://www.intel.com/programmable/technical-pdfs/683539.pdf>, 2021. Documento PDF. Accedido: 14-08-2025.
- [EE24] ESL-EPFL. X-heap. Recurso web: <https://github.com/esl-epfl/x-heap>, 2024. Repositorio GitHub. Accedido: 13-08-2025.
- [EE25] ESL-EPFL. X-heap documentation. Recurso web: <https://x-heap.readthedocs.io/en/latest/index.html>, 2025. Documentación de X-HEEP. Accedido: 13-08-2025.
- [Eng22] Semiconductor Engineering. High-level synthesis (hls). Recurso web: https://semiengineering.com/knowledge_centers/eda-design/verification/high-level-synthesis, 2022. Accedido: 14-08-2025.
- [EPF25a] EPFL. Logo de x-heap. Recurso web: <https://github.com/esl-epfl/x-heap/blob/main/docs/source/images/x-heap-outline.png>, 2025. Accedido: 10-08-2025.
- [EPF25b] EPFL. Source code of blinky over freertos. Recurso web: https://github.com/esl-epfl/x-heap/blob/main/sw/applications/example_freertos_blinky/main.c, 2025. Código de referencia FreeRTOS Blinky en X-HEEP. Accedido: 25-08-2025.
- [EPF25c] EPFL. Source code of timer sdk. Recurso web: https://github.com/esl-epfl/x-heap/blob/main/sw/applications/example_timer_sdk/main.c, 2025. Código de referencia del timer en X-HEEP. Accedido: 20-08-2025.

- [Esp] Real Academia Española. Definición de metodología. Recurso web: <https://dle.rae.es/metodología>. Página web de la Real Academia Española. Accedido 18-08-2025.
- [Eur24] Comisión Europea. Propuesta de modificación del reglamento (ue) 2021/2085. Recurso web: <https://eur-lex.europa.eu/legal-content/ES/TXT/HTML/?uri=CELEX:52022PC0047>, 2024. Accedido: 12-08-2025.
- [For06] Edaboard Forum. What does "wrapper" stand for? Recurso web: <https://www.edaboard.com/threads/what-does-wrapper-stand-for.63932/>, 2006. Accedido: 15-08-2025.
- [Fou25] RISC-V Foundation. Risc-v gnu toolchain. Recurso web: <https://github.com/riscv-collab/riscv-gnu-toolchain>, 2025. Repositorio de GCC para RISC-V. Accedido: 18-08-2025.
- [Fre25] FreeRTOS. Página web de freertos. Recurso web: <https://www.freertos.org/>, 2025. Accedido: 10-08-2025.
- [Gro21] OpenHW Group. Obi v1.2. Recurso web: <https://raw.githubusercontent.com/openhwgroup/obi/188c87089975a59c56338949f5c187c1f8841332/OBI-v1.2.pdf>, 2021. Documento PDF. Accedido: 29-08-2025.
- [Gro23] OpenHW Group. Cv32e20/cv32e40* user manual. Recurso web: https://docs.openhwgroup.org/projects/cve2-user-manual/en/latest/01_specification/index.html, 2023. Accedido: 25-08-2025.
- [HH21] S. Harris and D. Harris. *Digital Design and Computer Architecture, RISC-V Edition*. Elsevier Science, 2021.
- [IEE01] IEEE. Ieee standard verilog hardware description language. Recurso web: <https://standards.ieee.org/ieee/1364/2052/>, 2001. Accedido: 14-08-2025.
- [Int21] RISC-V International. About risc-v. Recurso web: <https://riscv.org/about/>, 2021. Accedido: 12-08-2025.
- [ISD23] ISDI. Rtl verilog. Recurso web: <https://www.doulos.com/knowhow/verilog/rtl-verilog/>, 2023. Accedido: 14-08-2025.
- [Jac00] Rumbaugh Jacobson, Booch. *El proceso unificado de desarrollo de software*. Addison Wesley, second edition, July 2000. ISBN: 978-8478290369.
- [Jen22] Jonas Julian Jensen. How the axi-style ready/valid handshake works. Recurso web: <https://vhdlwhiz.com/how-the-axi-style-ready-valid-handshake-works/>, 2022. Accedido: 25-08-2025.

- [Jim23] Ormarys Jimenez. ¿qué es diseño de software y hardware? Recurso web: <https://www.iutepi.edu/que-es-diseno-de-software-y-hardware/#:~:text=En%20pocas%20palabras%2C%20se%20refiere%20al%20proceso,PCB%2C%20cables%2C%20resistencias%2C%20capacitores%20y%20mucho%20m%C3%A1s>, 2023. Accedido: 12-08-2025.
- [Mic14] Microchip. 9 reasons why the vivado design suite accelerates design productivity. Recurso web: https://www.xilinx.com/publications/prod_mktg/vivado/Vivado_9_Reasons_Background.pdf, 2014. Accedido: 14-08-2025.
- [Mic24] Microchip. Amd vivado design suite: Redefining fpga and soc development. Recurso web: <https://www.microchipusa.com/articles/amd/amd-vivado-design-suite-redefining-fpga-and-soc-development>, 2024. Accedido: 14-08-2025.
- [Mic25] Microsoft. Download visual studio code. Recurso web: <https://code.visualstudio.com/download>, 2025. Página de descarga de Visual Studio Code. Accedido: 18-08-2025.
- [MK04] C. Maxfield and I. Kerszenbaum. *The design warrior's guide to FPGAs*. Elsevier, 2004.
- [MSM⁺24] Simone Machetti, Pasquale Davide Schiavone, Thomas Christoph Müller, Miguel Peón-Quirós, and David Atienza. X-heep: An open-source, configurable and extendible risc-v microcontroller for the exploration of ultra-low-power edge accelerators, 2024.
- [Ope25] OpenTitan. Register generator reggen and regtool. Recurso web: https://opentitan.org/book/util/reggen/doc/setup_and_use.html#register-generator-reggen-and-regtool, 2025. Uso de reggen y regtool. Accedido: 20-08-2025.
- [Pro25] Zephyr Project. Página web de zephyr. Recurso web: <https://www.zephyrproject.org/>, 2025. Accedido: 10-08-2025.
- [Qui25a] David Mallasén Quintana. Adding a simple counter peripheral through the x-aif. Recurso web: https://campusvirtual.uclm.es/pluginfile.php/1069108/mod_resource/content/5/Memoria_caso_practico_5_XAIF.pdf, 03 2025. Documento PDF de la práctica 5. Accedido: 15-08-2025.
- [Qui25b] David Mallasén Quintana. Gr-heep connect bus branch. Recurso web: <https://github.com/davidmallasen/GR-HEEP/tree/connect-bus>, 03 2025. Rama de GR-HEEP para integración del acelerador. Accedido: 15-08-2025.

- [Qui25c] David Mallasén Quintana. Simple counter module. Recurso web: https://github.com/Integrated-Systems-Architecture/simple_cnt, 2025. Repositorio de contador integrado en GR-HEEP. Accedido: 15-08-2025.
- [Rea20] RealDigital. A brief history of verilog. Recurso web: <https://www.realdigital.org/doc/1946d210d1411b214203a6673322a61f>, 2020. Accedido: 14-08-2025.
- [Rin24] Fernando Rincón. Microsof stream - metodología. Recurso web: https://pruebasaluuclm-my.sharepoint.com/personal/fernando_rincon_uclm_es/_layouts/15/stream.aspx?id=%2Fpersonal%2Ffernando%5Frincon%5Fuclm%5Fes%2FDocuments%2Fmaster%2Fmetodologia%2Emp4&ga=1&referrer=StreamWebApp%2EWeb&referrerScenario=AddressBarCopied%2Eview%2E121e8790%2Daeb0%2D4659%2Da548%2Dc3808491aa3a, 2024. Vídeo de metodologías EDA. Accedido: 14-08-2025.
- [Rom19] David Romano. A brief history of fpga. Recurso web: <https://makezine.com/article/maker-news/a-brief-history-of-fpga/>, 2019. Accedido: 11-08-2025.
- [Sch02] R.J Schweers. Metodologías de diseño de hardware. Recurso web: https://sedici.unlp.edu.ar/bitstream/handle/10915/3835/2_-_Metodolog%C3%ADas_de_dise%C3%B1o_de_hardware.pdf?sequence=4&isAllowed=y, 2002. Documento PDF, Universidad de la Plata. Accedido: 10-08-2025.
- [SER22] Cadena SER. La cátedra perte-chip llega a la uclm para la formación de expertos en microelectrónica y semiconductores. Recurso web: <https://cadenaser.com/castillalamancha/2024/10/04/la-catedra-perte-chip-llega-a-la-uclm-para-la-formacion-de-expertos-en> 2022. Accedido: 12-08-2025.
- [Sha23] Vishal Sharma. Fpga development made easy: A comprehensive guide to vivado and vitis hls. Recurso web: https://medium.com/@the_daft_introvert/fpga-development-made-easy-a-comprehensive-guide-to-vivado-and-vitis-h 2023. Accedido: 14-08-2025.
- [Sta99] Richard Stallman. make - gnu make utility to maintain groups of programs. Recurso web: <https://linux.die.net/man/1/make>, 1999. Manual de make. Accedido: 18-08-2025.
- [Tec24] TechTarget. What is a real-time operating system (rtos)? Recurso web: <https://www.techtarget.com/searchdatacenter/definition/real-time-operating-system>, 2024. Accedido: 14-08-2025.

- [Tor24] VLSI Lab Politecnico Di Torino. Gr-heap, a fork of x-heap. Recurso web: <https://github.com/esl-epfl/x-heap>, 2024. Repositorio de GR-HEEP. Accedido: 13-08-2025.
- [Tos22] Frank Toshioka. Beneficios del hls. Recurso web: https://es.linkedin.com/advice/1/what-advantages-disadvantages-using-high-level?lang=es&lang=es&contributionUrn=urn%3Ali%3Acomment%3A%28articleSegment%3A%28urn%3Ali%3AlinkedInArticle%3A7046538086146580481%2C7046538089065787392%29%2C7169942824606535681%29&articleSegmentUrn=urn%3Ali%3AarticleSegment%3A%28urn%3Ali%3AlinkedInArticle%3A7046538086146580481%2C7046538089065787392%29&dashContributionUrn=urn%3Ali%3Afsd_comment%3A%287169942824606535681%2CarticleSegment%3A%28urn%3Ali%3AlinkedInArticle%3A7046538086146580481%2C7046538089065787392%29%29, 2022. Accedido: 14-08-2025.
- [WH15] N.H.E. Weste and D. Harris. *CMOS VLSI Design : A circuits and systems perspective*. Pearson, 2015.
- [Wik23] Wikipedia. Hardware description language. Recurso web: https://en.wikipedia.org/wiki/Hardware_description_language, 2023. Accedido: 14-08-2025.
- [Win23] Windriver. What is a real-time operating system (rtos)? Recurso web: <https://www.windriver.com/solutions/learning/rtos>, 2023. Accedido: 14-08-2025.
- [Xat21] Xataka. El primer chip risc-v europeo cobra vida: Epac está destinado a supercomputadoras, pero esto es solo el principio. Recurso web: <https://www.xataka.com/componentes/primer-chip-risc-v-europeo-cobra-vida-epac-esta-destinado-a-supercomputadoras> 2021. Accedido: 12-08-2025.
- [Xil23] Xilinx. Vivado design suite user guide: Designing with ip (ug896). Recurso web: <https://docs.amd.com/r/2023.2-English/ug896-vivado-ip/IP-Integrator>, 2023. Accedido: 16-08-2025.